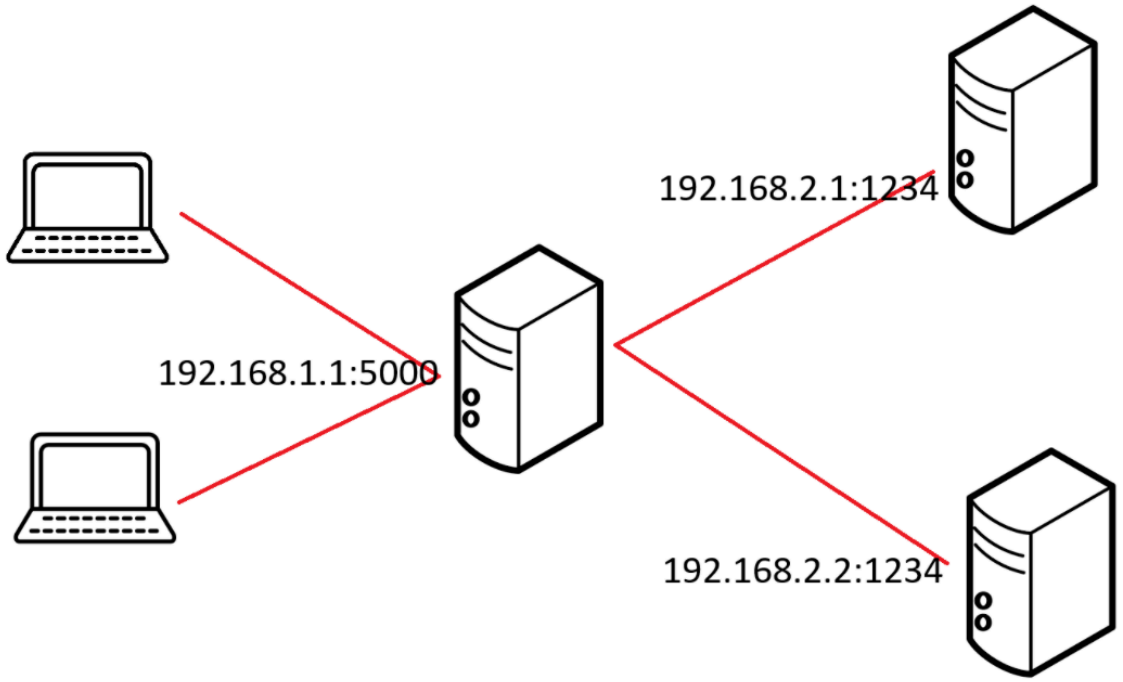
LIMBONET

Limbonet es una plataforma de gestión centralizada de configuración de servicios de red a nivel de sistema operativo en nodos Linux, basada en arquitectura cliente-servidor, orientada a la generación y despliegue de configuraciones de servicios como DHCP y DNS



El sistema permite configurar manualmente los puertos de escucha, del WebGUI y del servicio agente.

Limbonet se compone de dos componentes principales:

- Limbonet WebGUI: interfaz central de gestión
    - Genera los ficheros de configuración a partir de la interfaz web, estos son enviados posteriormente al servicio agente del nodo correspondiente para su aplicación
- Limbonet Service: agente desplegado en los nodos gestionados
    - Aplica los cambios.

La WebGUI genera las configuraciones y el agente las aplica en el servidor final.

Se recomienda el uso de firewall y/o segmentación de red (VPN o Red Privada Virtual) para proteger la comunicación entre el WebGUI y los nodos

Introducción y enfoque

LIMBONET pretende facilitar la gestión de servicios de red en entornos Linux, a través de una interfaz web que permita la generación, validación y despliegue de configuraciones de forma remota. La plataforma trata de simplificar tareas informáticas que, para realizarse, se necesitaban conocimientos avanzados de sistemas.

LIMBONET está pensado para PYMES que no disponen de departamento informático o con recursos técnicos limitados. También puede ser de utilidad en entornos educativos, laboratorios, y organizaciones que deseen administrar de forma más sencilla los servidores.

Un gran número de PYMES carecen de personal especializado en administración de sistemas, siendo habitual el uso de servicios de red integrados en el router, con capacidades limitadas de configuración. LIMBONET permite a los usuarios disponer de DNS y DHCP sin necesidad de tener conocimientos avanzados.

Los servicios de red se suelen configurar editando manualmente los ficheros de configuración, puede ocasionar errores que afecten a toda la red

El sistema resuelve este problema proporcionando una plataforma centralizada que permite generar configuraciones de forma visual, validar los cambios antes de aplicarlos y desplegarlos automáticamente en los servidores correspondientes.

Entre las principales ventajas que ofrece LIMBONET se encuentran:

- Administración centralizada de múltiples servidores
- Reducción de errores de configuración manual
- Interfaz gráfica sencilla e intuitiva
- Independencia respecto al router utilizado
- Mayor control
- Validación automática de configuraciones antes de su despliegue
- Posibilidad de escalar la infraestructura conforme aumenten las necesidades de la organización.

Actualmente la plataforma permite la gestión centralizada de servicios DNS y DHCP sobre sistemas Linux mediante una arquitectura cliente-servidor. También incorpora mecanismos de autenticación, registro de eventos y monitorización de servicios.

Conceptos fundamentales:

- Servidor DNS: Servidor encargado de traducir nombres de dominio fáciles de recordar en direcciones IP comprensibles por los equipos de red, por ejemplo empresa.com a 192.168.1.1. Entre sus parámetros:
    - Zona DNS directa: permite resolver nombres de dominio hacia direcciones IP
    - Zona DNS inversa: realiza el proceso contrario, permitiendo obtener un nombre de host a partir de una dirección IP
- Servidor DHCP: Asigna automáticamente parámetros de red a los dispositivos que se conectan a una red, evitando que el administrador tenga que configurar manualmente cada equipo. Entre sus parámetros:
    - Puerta de enlace (Gateway): Es el dispositivo encargado de permitir la configuración entre redes, normalmente correspondiendo al router. Cuando debe comunicarse con una dirección fuera de su red local, envía el tráfico a la puerta de enlace para que lo enrute hacia su destino
    - Máscara de red: determina que parte de una dirección IP identifica la red y que parte a los equipos conectados a ella, por ejemplo: 255.255.255.0, los tres primeros octetos representan la red y el último identifica al equipo

    - Clase DHCP y Pool DHCP: Una clase DHCP permite agrupar dispositivos con características comunes, junto con las pool DHCP para aplicar configuraciones específicas, por ejemplo, una clase para impresoras, otra para cámaras IP…
        - Una clase DHCP permite agrupar dispositivos con características similares.
        - Una pool DHCP permite asignar rangos de direcciones distintas a cada grupo de dispositivos.

Una VLAN permite dividir una red física en varias redes lógicas independientes. Los usuarios de una VLAN no pueden comunicarse directamente con otra salvo que exista un sistema determinado, y cada VLAN puede disponer de su propio ámbito DHCP

La modificación de la configuración de red del sistema operativo es una operación crítica, una configuración incorrecta puede provocar pérdida de conectividad, acceso remoto, conflictos, interrupción de servicios… por este motivo la funcionalidad se considera experimental, y está orientada a entornos de laboratorio

Arquitectura

La arquitectura de LIMBONET ha sido diseñada siguiendo criterios de seguridad, facilidad de mantenimiento y escalabilidad. El sistema separa la producción de su aplicación en lols servidores de destino, permitiendo reducir los riesgos de seguridad.

DHCP y DNS pueden estar en un mismo servidor, sin embargo mantenerlos en diferentes servidores proporcionan las siguientes ventajas:

- Menor impacto ante fallos
- Mejor distribución de recursos
- Mayor facilidad a la hora de mantenimiento
- Posibilidad de escalar cada servicio de forma independiente
- Mayor seguridad mediante segmentación

Los servicios que incluyen los routers domésticos o en pequeñas empresas suelen ser limitados con escasas opciones de configuración y poca capacidad de monitorización

El servidor WebGUI puede ejecutarse en una Raspberry Pi debido a los bajos requisitos de recursos de la aplicación, permitiendo minimizar el costo de hardware y el consumo energético.

Flask ha sido elegido por ser un framework ligero y flexible, al igual que FastAPI especializado a la hora de las llamadas API.

Seguridad

La seguridad es fundamental en LIMBONET, debido a que la plataforma administra servicios críticos para el funcionamiento de la red.

Se recomienda proteger tanto el servidor WebGUI como los nodos gestionados mediante reglas del firewall, permitiendo solamente el tráfico estrictamente necesario.

Cada servicio debe exponer únicamente los puertos requeridos para su funcionamiento. Los servicios DNS y DHCP deben estar disponibles únicamente para las redes autorizadas de la organización.

La interfaz está publicada con HTTPS protegiendo las credenciales y la información intercambiada entre usuarios y servidores, mientras que las reglas del firewall permiten controlar que sistemas pueden acceder a cada servicio

Los datos introducidos son validados antes de ser presentados. Una mala configuración DNS y DHCP puede provocar la caída del servidor, por ello las configuraciones son verificadas antes de ser aplicadas.

Funcionamiento principal:

El sistema se compone de las siguientes tecnologías:

- Flask y Jinja2 para la interfaz web
- FastAPI para la API del servicio agente
- Bcrypt para el hash de contraseñas
- Requests para comunicación HTTP entre componentes

Limbonet-webgui se ejecuta como un servidor web en el servidor principal, accesible por los usuarios. Al acceder a la interfaz web, y a las respectivas configuraciones, a través de un formulario se procesan los parámetros introducidos por el usuario para generar el fichero de configuración..

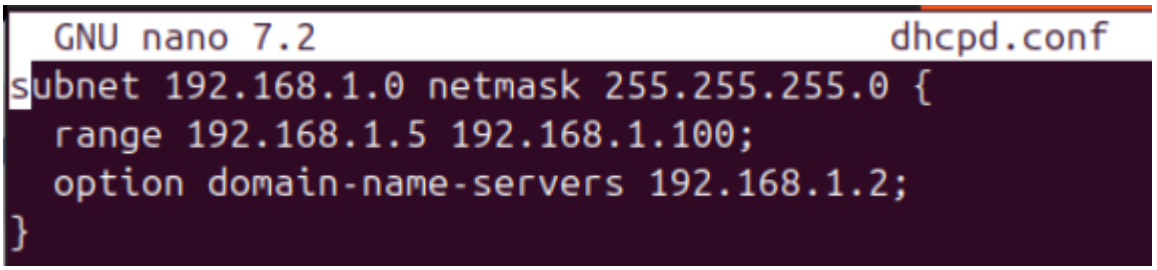
El WebGUI no aplica cambios directamente en los nodos, sino que actúa como sistema de orquestación, generando configuraciones que son enviadas al servicio agente correspondiente para su ejecución

Ejemplo de flujo:

El usuario genera una configuración desde WebGUI, esta se envía al Service del nodo seleccionado, donde se valida la API Key y se aplica la configuración.



Tras aplicar los cambios:

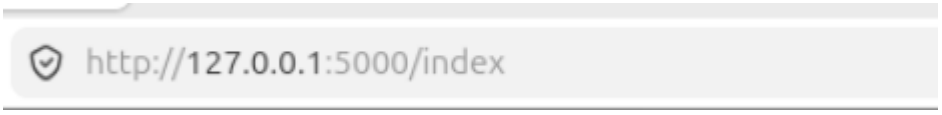


Se genera este fichero

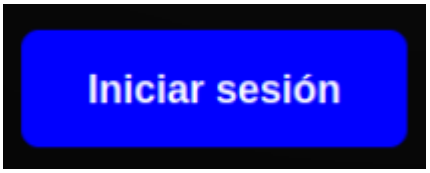
Limbonet-service se ejecuta en los servidores finales y este aplica los cambios cuando recibe las peticiones de los servidores con limbonet-webgui

1. Usuario configura parámetros en WebGUI
2. WebGUI genera fichero de configuración
3. El fichero se envía al servicio agente del nodo
4. El servicio valida la API Key asociada previamente al nodo registrado
5. El servicio aplica la configuración localmente

Modo de uso



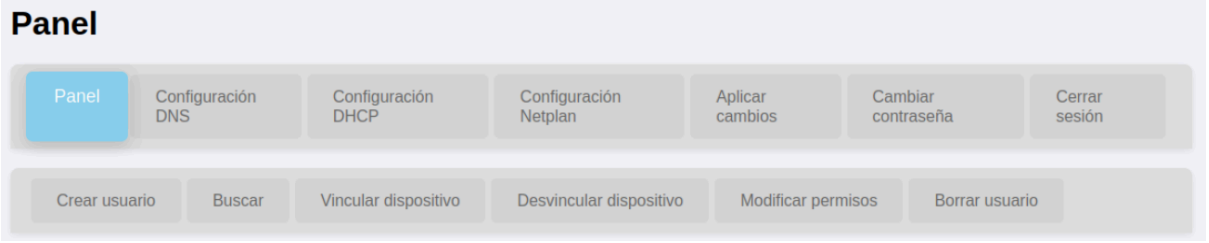
Se puede acceder con https://127.0.0.1:5000 a la interfaz web de Limbonet-Webgui



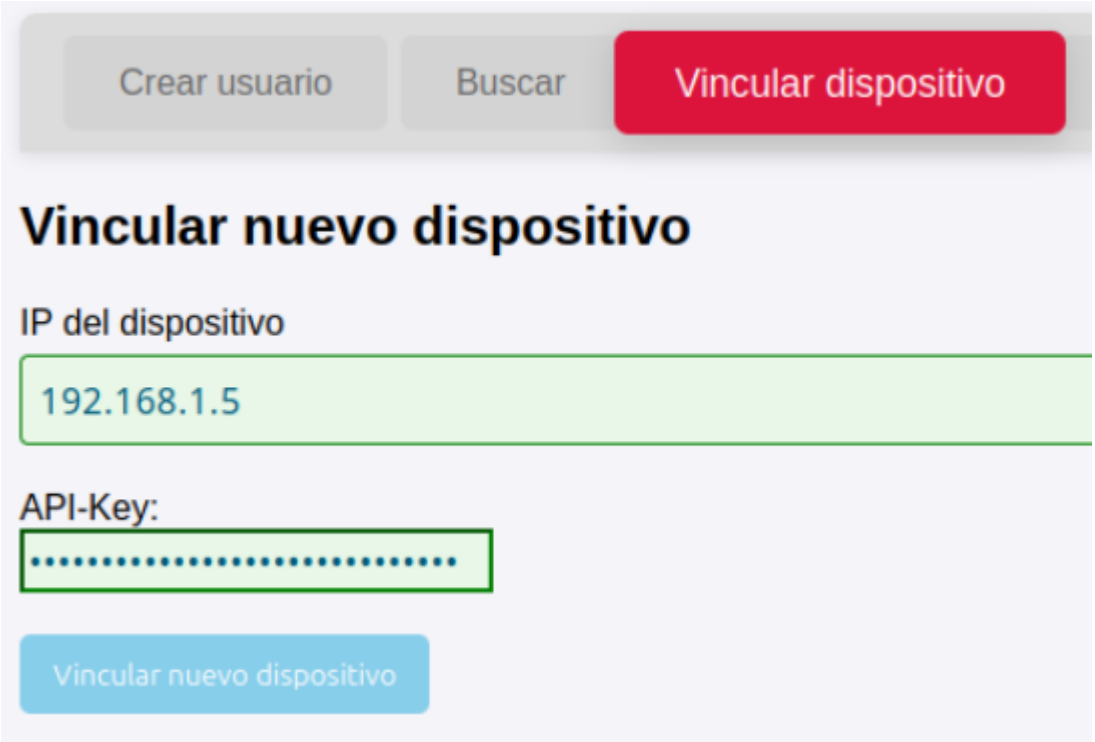
Se hace clic en el botón de iniciar sesión



El usuario y contraseña por defecto es "admin", se recomienda cambiarlo nada más entrar.



Dentro del panel para agregar un dispositivo vamos a "Vincular dispositivo".



Durante el proceso del registro del nodo se introduce la IP del servicio.
La API Key es generada en el nodo durante la instalación del agente y utilizada para autenticar las peticiones del WebGUI.

```
  GNU nano 7.2                                  config.json *
{
        "api_key":"pon_tu_api_key_aqui",
        "PORT": "1234",
        "TEMP_DIR": "./tmp",
        "DHCP_DIR": "./dhcp",
        "DNS_DIR": "./dns",
        "NETPLAN_DIR": "./netplan"
}
```

Esto es visible desde el fichero de configuración de Limbonet-service.
Una vez hecho esto, se puede usar directamente de su interfaz web.

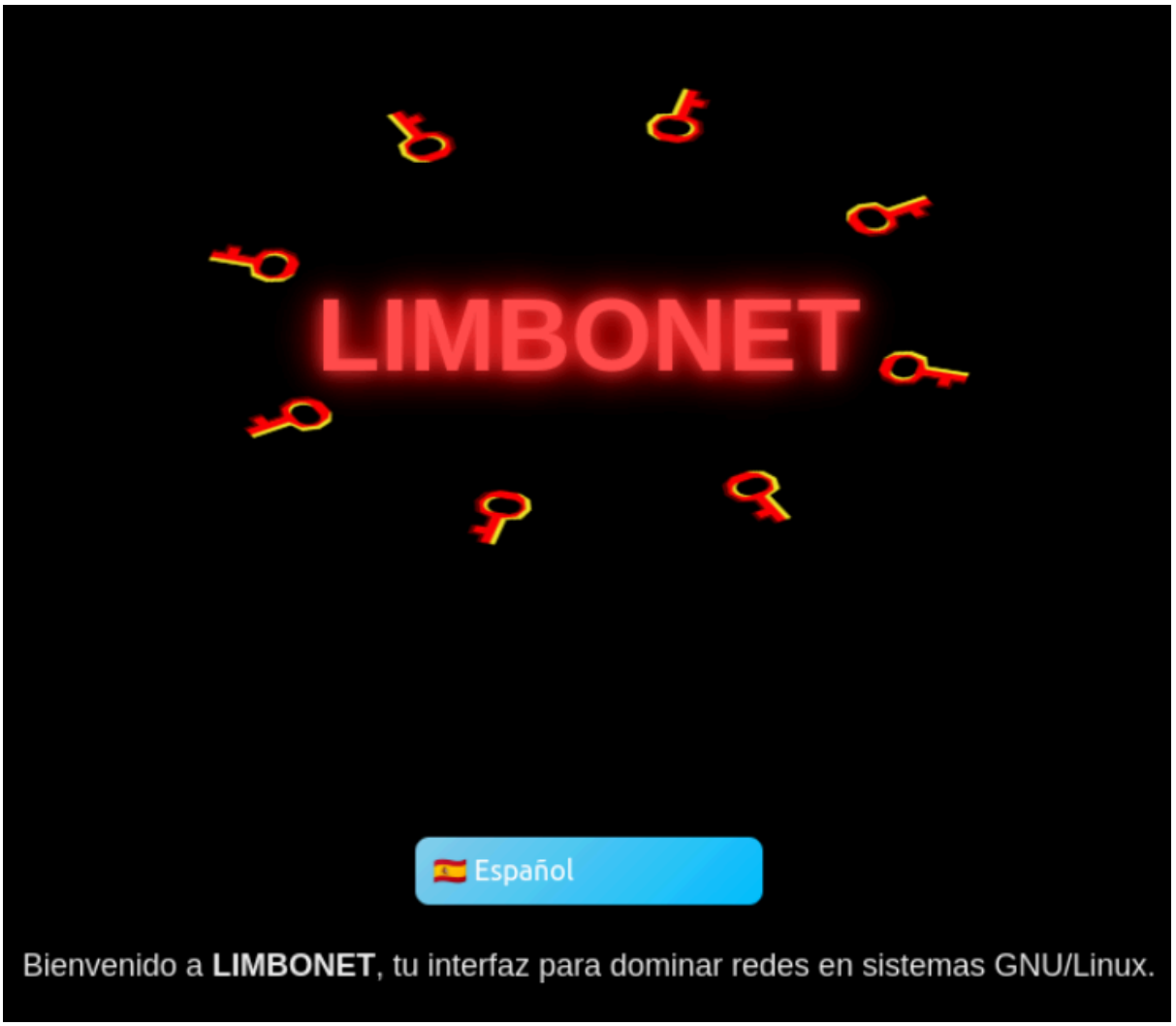
Diseño de Limbonet webgui

Voy a dividir la explicación en dos partes: el frontend y el backend.

Frontend

```
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.104.199.35:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 593-086-914
```

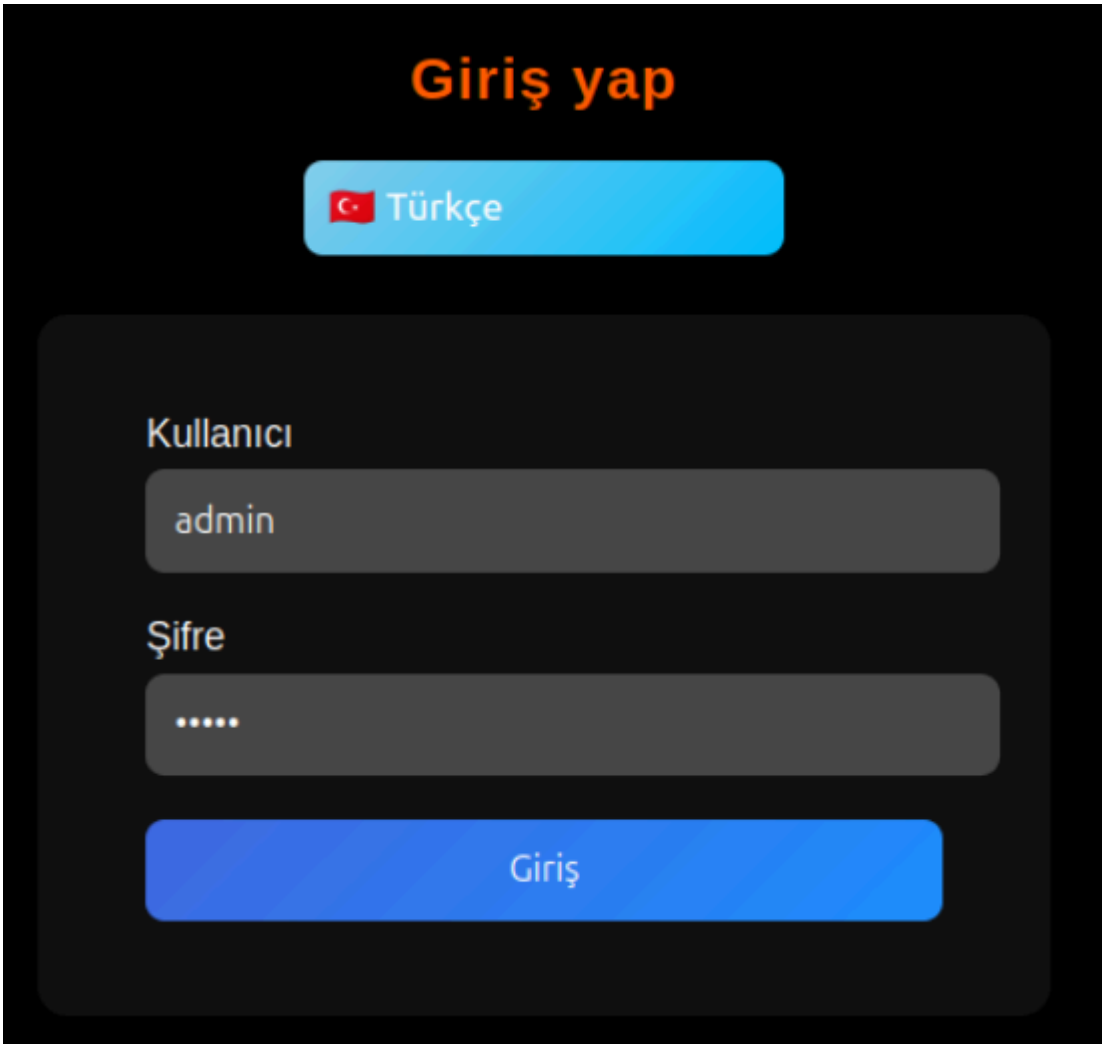
El despliegue puede realizarse utilizando un servidor WSGI como Apache o equivalente.
Una vez que se arranca, se abre por defecto en todas las interfaces en el puerto 5000



Al entrar, se llega a la pantalla principal.
Se puede cambiar el idioma con el menú



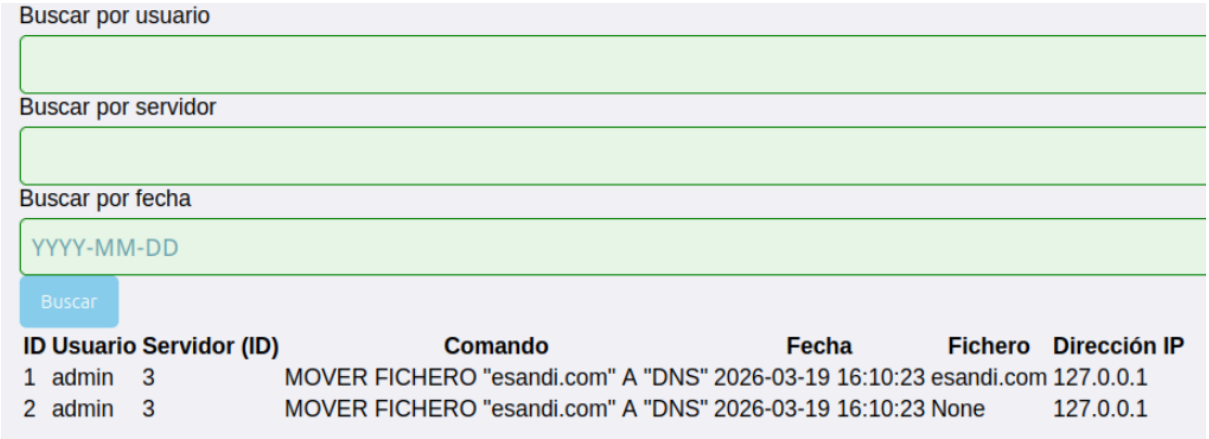
Hay dos opciones, iniciar sesión o acceder al repositorio de GitHub. Se hará clic en Iniciar sesión



En mi caso, inicio sesión con el usuario administrador, que su usuario y contraseña por defecto es "admin". Se recomienda cambiarlo nada más acceder.

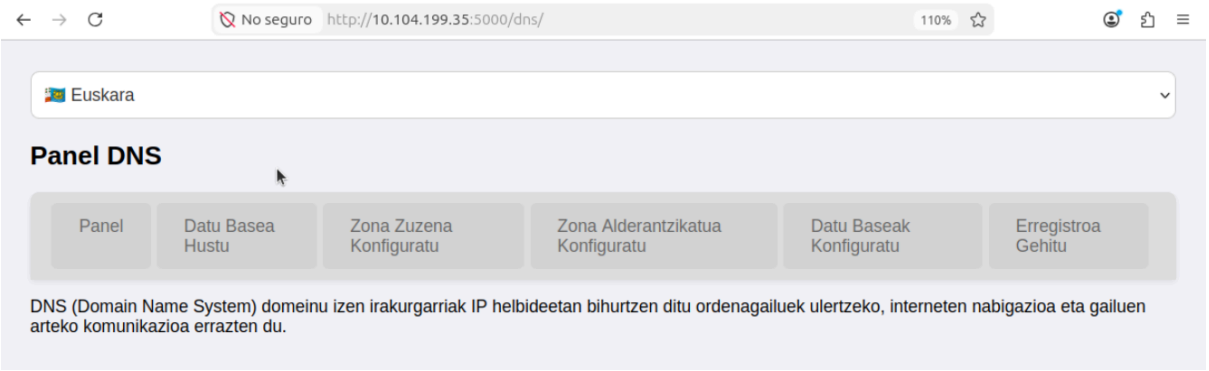


Esta es la vista del panel principal. En el caso de no ser una cuenta con permisos administrativos, la barra inferior no aparecerá.



También hay un menú para poder revisar los logs

Menú principal

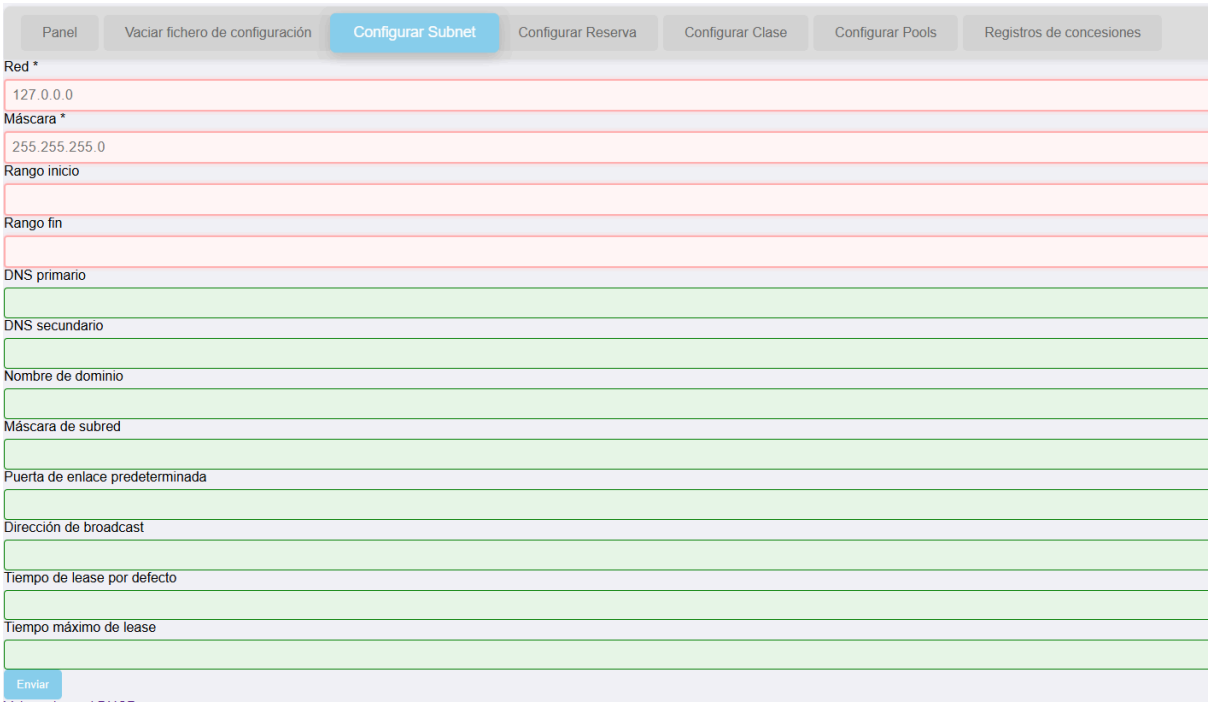


Si selecciono cualquier opción del menú, me llevará a su correspondiente página con más opciones

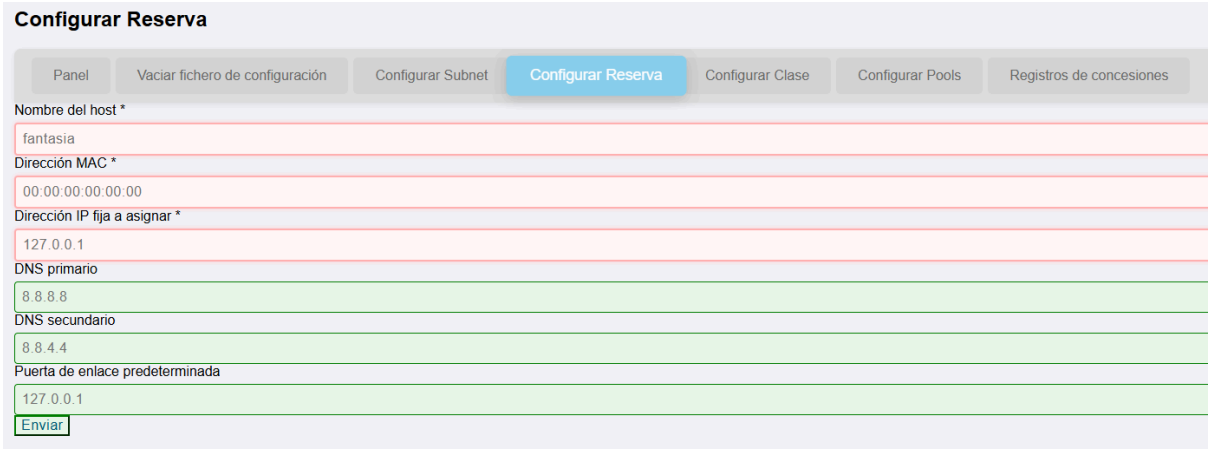
Menú de DHCP

DHCP es un servicio que asigna automáticamente direcciones IP a los dispositivos de una red, gracias a esto, no es necesario configurar manualmente cada ordenador, impresora, cámara…

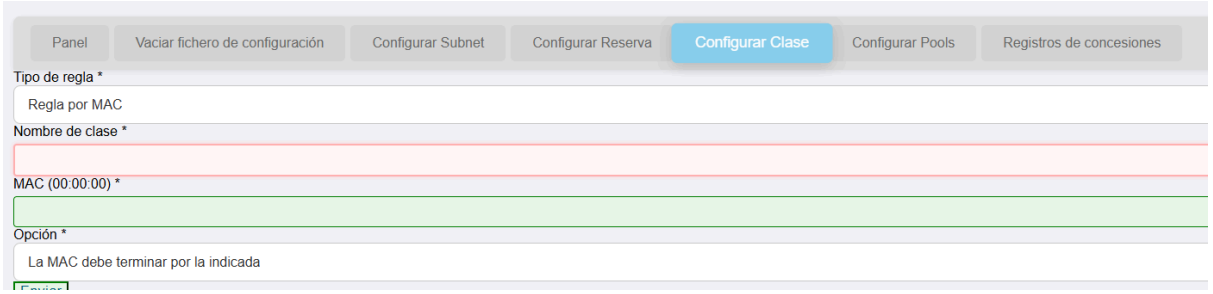
- Ámbito DHCP
    - Conjunto de direcciones IP que el servidor DHCP puede repartir dentro de una red determinada. Se puede imaginar como un edificio, el ámbito es el edificio y las direcciones IP son los apartamentos disponibles.
    - Por ejemplo: Una empresa tiene la red 192.168.1.0/24, y el administrador decide que DHCP asignará direcciones desde 192.168.1.100 hasta 192.168.1.200
- Reserva DHCP
    - Una reserva DHCP permite que un dispositivo concreto reciba siempre la misma dirección IP, identificado por la dirección física (MAC) y le entrega siempre la IP reservada.
    - Por ejemplo: La impresora principal siempre debe ser accesible en la misma dirección
- Clase
    - Una clase DHCP es una forma de agrupar dispositivos que comparten características comunes para aplicarles configuraciones determinadas.
    - Por ejemplo: se pueden crear clases para determinados dispositivos (cámaras IP, impresoras…) debido a que cada fabricante usa una dirección MAC determinada
- Pool
    - Un pool es un grupo o rango de direcciones IP que DHCP utiliza para asignarlas a determinados dispositivos, se pueden usar diferentes tipos de pools para organizar mejor la red
    - Por ejemplo:
        - Pool para usuarios: 192.168.1.100 - 192.168.1.180
        - Pool para impresoras: 192.168.1.181 - 192.168.1.190



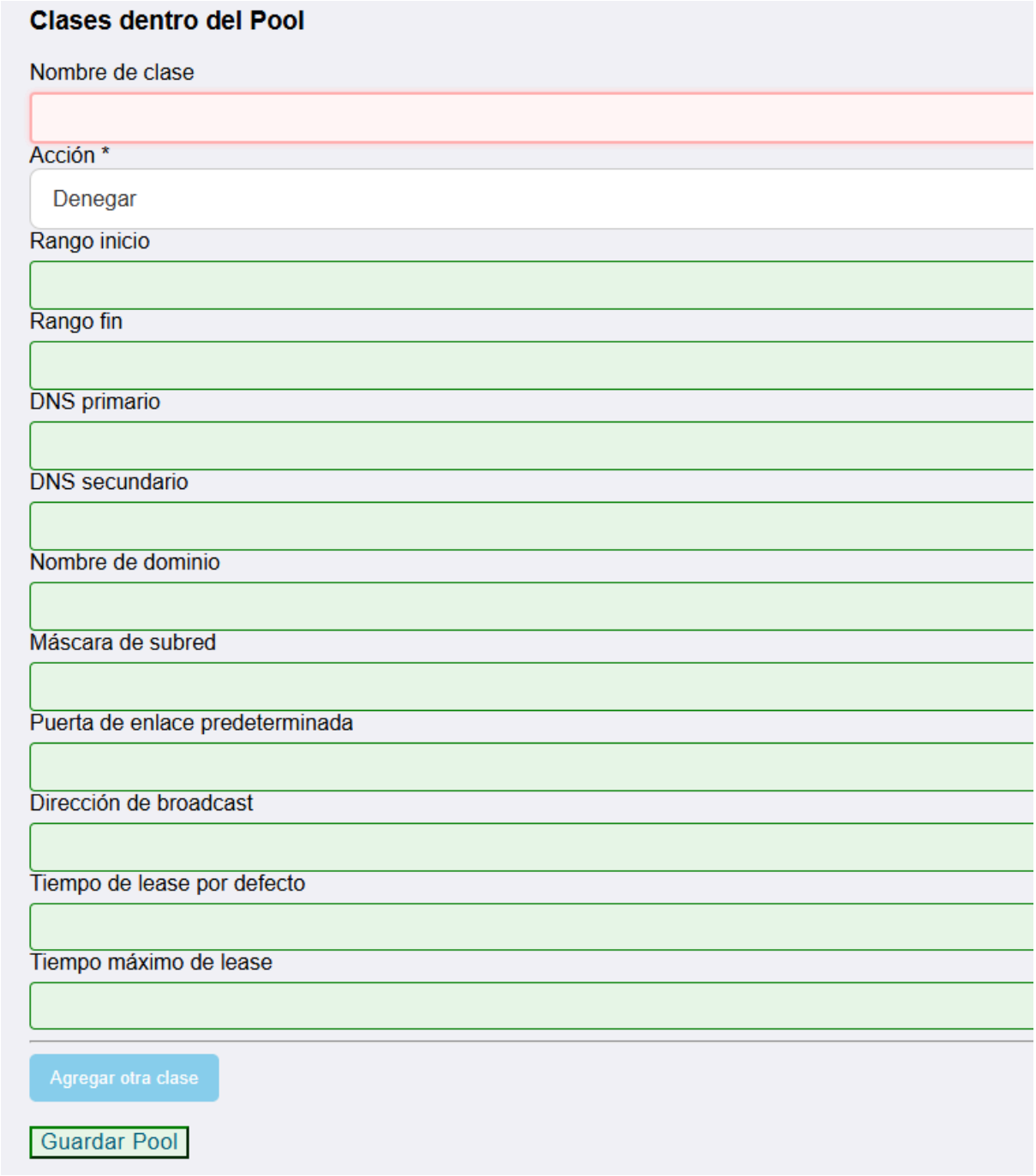
Ejemplo de la configuración de subnet, primero se especifica la red, posteriormente la máscara, rango de inicio y fin, y finalmente las configuraciones de red de los equipos que se conecten.



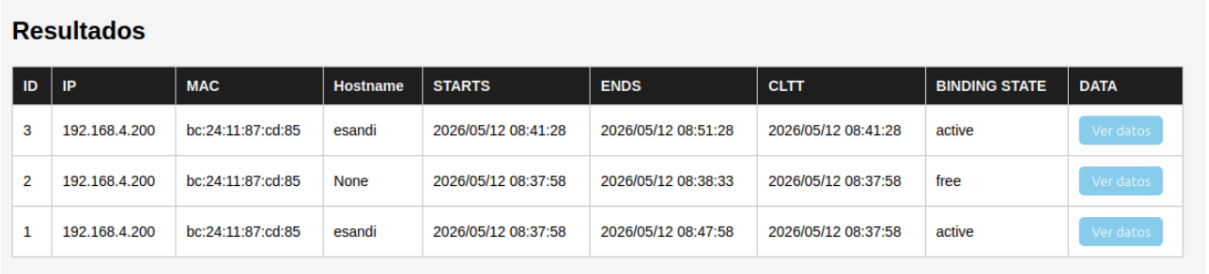
A la hora de configurar una reserva, se agrega la configuración de la dirección MAC, y la dirección IP a asignar. Opcionalmente se puede modificar su servidor DNS y puerta de enlace predeterminada



A la hora de configurar clases, se puede definir una clase con una regla de dirección MAC debido a que, dependiendo del fabricante, tiene una dirección específica.



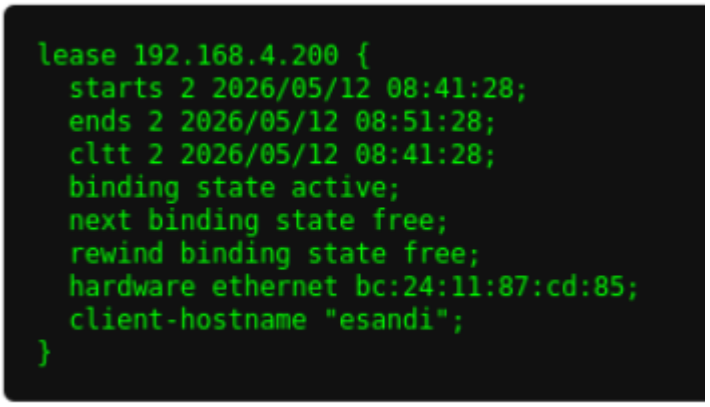
A la hora de configurar los Pools, se debe establecer una clase y una acción de, a los usuarios que pertenecen o no pertenecen a la clase, unas determinadas reglas.



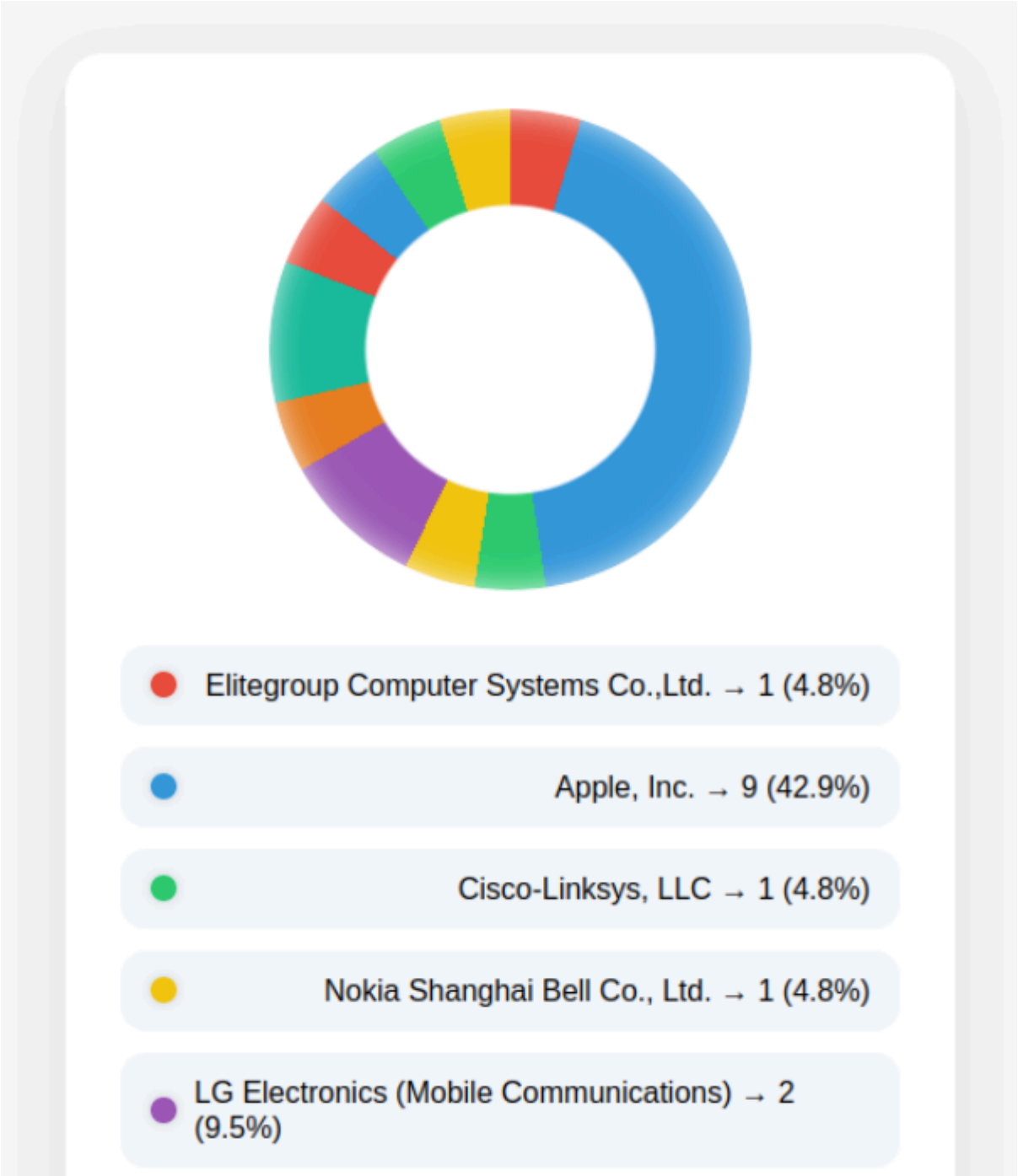
Resultados

ID	IP	MAC	Hostname	STARTS	ENDS	CLTT	BINDING STATE	DATA
3	192.168.4.200	bc:24:11:87:cd:85	esandi	2026/05/12 08:41:28	2026/05/12 08:51:28	2026/05/12 08:41:28	active	Ver datos
2	192.168.4.200	bc:24:11:87:cd:85	None	2026/05/12 08:37:58	2026/05/12 08:38:33	2026/05/12 08:37:58	free	Ver datos
1	192.168.4.200	bc:24:11:87:cd:85	esandi	2026/05/12 08:37:58	2026/05/12 08:47:58	2026/05/12 08:37:58	active	Ver datos

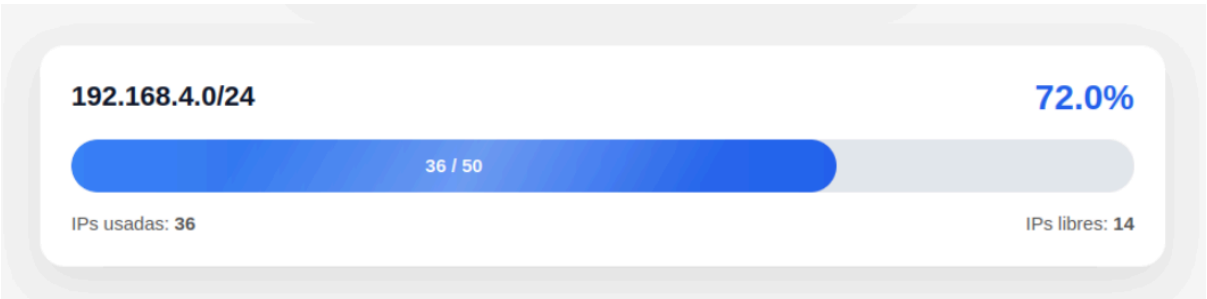
Dentro se pueden ver las concesiones de una forma más eficiente, buscando por hostname, direcciones IP o fechas.



También se puede ver la concesión con más detalle.



Dentro del menú se pueden ver estadísticas de los fabricantes de los dispositivos que están conectados.

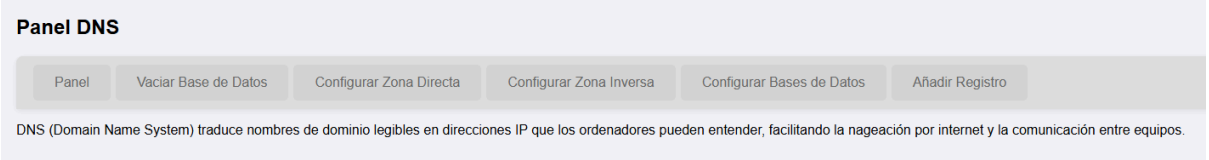


Y también muestra por cada subnet un porcentaje de uso.

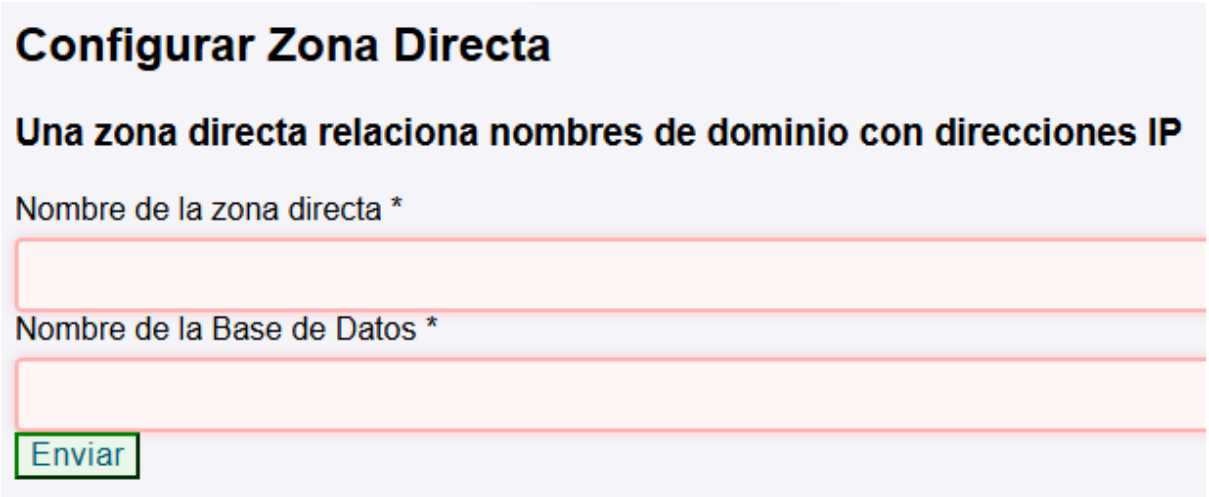
Menú de DNS

El servicio DNS permite traducir nombres fáciles de recordar en direcciones IP y viceversa. Gracias a DNS, los usuarios pueden acceder a equipos y servicios usando nombres en vez de números.

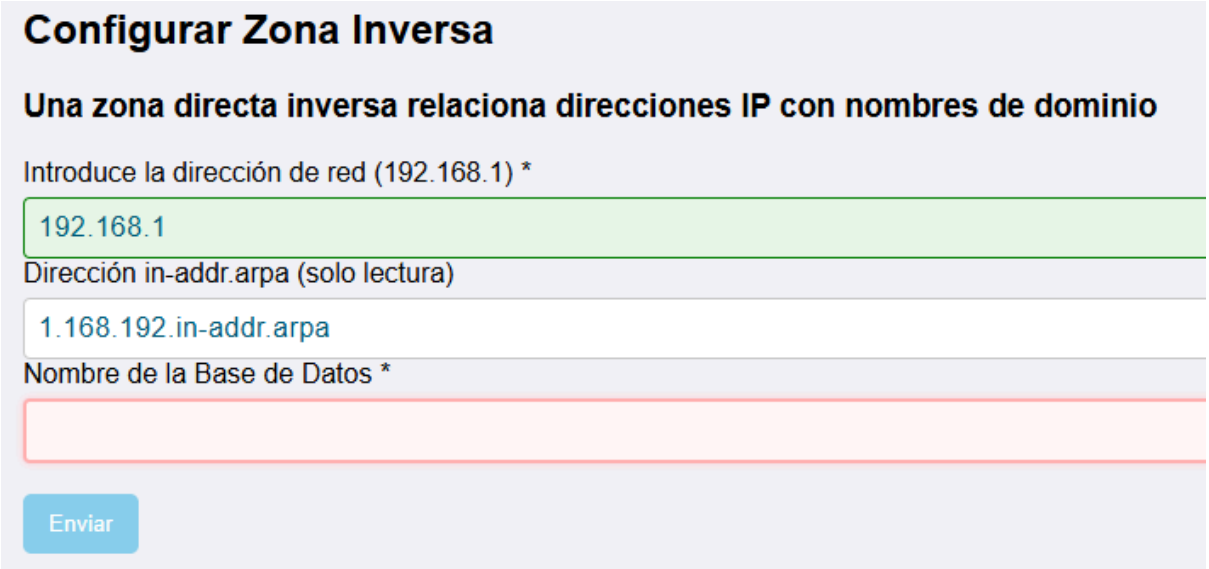
- Zona directa
  - Se utiliza para convertir un nombre de dominio en una dirección IP, por ejemplo:
  - Empresa.local lo transforma en 192.168.1.4
  - Esto funciona para acceder a servidores mediante nombres fáciles de recordar.
- Zona inversa
  - La zona inversa realiza la operación contraria a la zona directa, transforma una dirección IP a un nombre. Cuando una aplicación conoce la IP 192.168.1.10, puede consultar el DNS para averiguar que corresponde a servidor.empresa.local
- Ficheros DB
  - Los ficheros DB (Bases de datos) almacenan los registros DNS de una zona.



Este es el menú principal del panel DNS

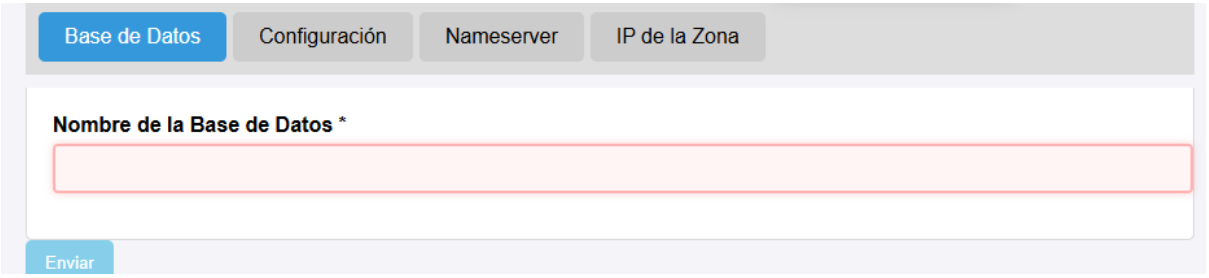


A la hora de configurar una zona directa, pide el nombre de la zona y el nombre del fichero DB.
El nombre de la zona directa corresponde al dominio que se va a administrar en el servidor DNS, representando el conjunto de nombres que pertenecen a una organización, departamento o red, por ejemplo, si se usa empresa.local, otros nombres pueden ser www.empresa.local, correo.empresa.local… de modo que la zona se llamará "empresa.local"

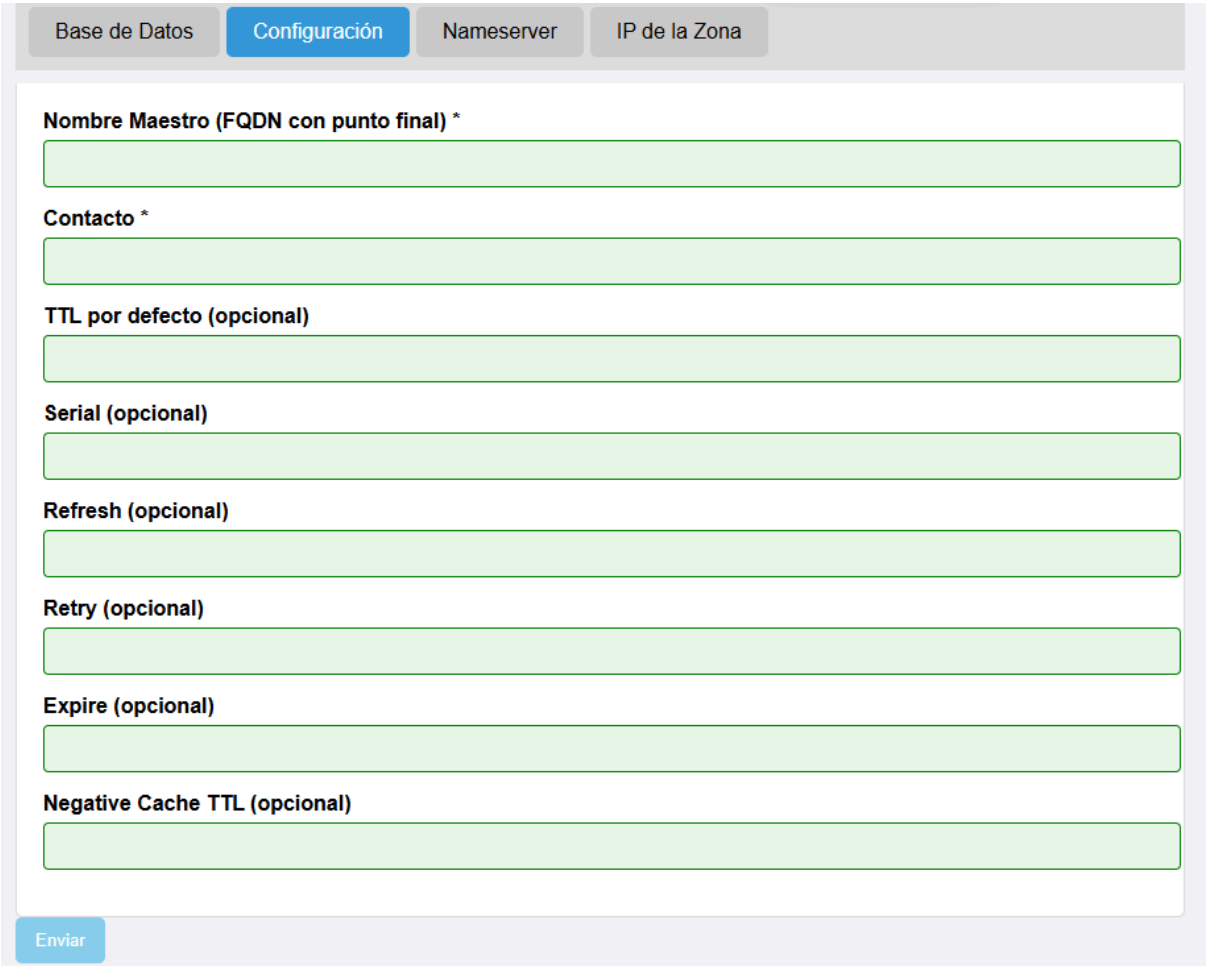


A la hora de configurar una zona inversa, pide el nombre de la zona y el nombre del fichero DB. Este representa un rango de direcciones IP administrado por el servidor DNS, y se debe utilizar una representación especial basada en la red IP.
Se usa un dominio in-addr.arpa para las resoluciones inversas, estableciendo ahí la dirección de la red. Se debe introducir el prefijo de la dirección de red, y este es automáticamente transformado a dirección in-addr.arpa.

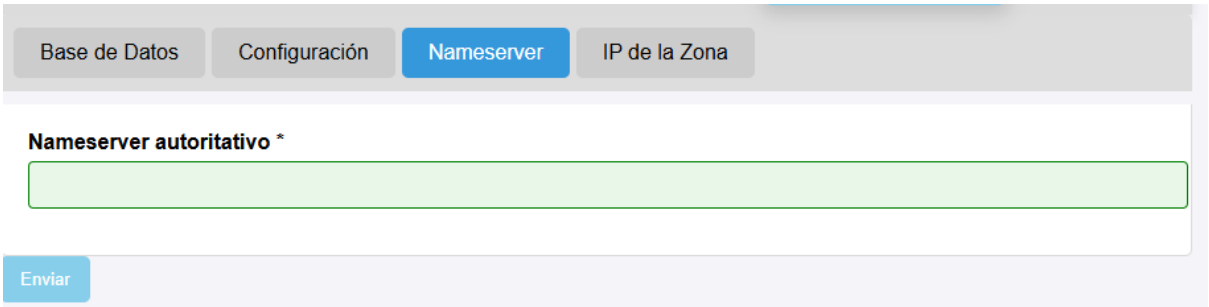
A la hora de configurar un fichero DB



Primero se escribe el nombre.



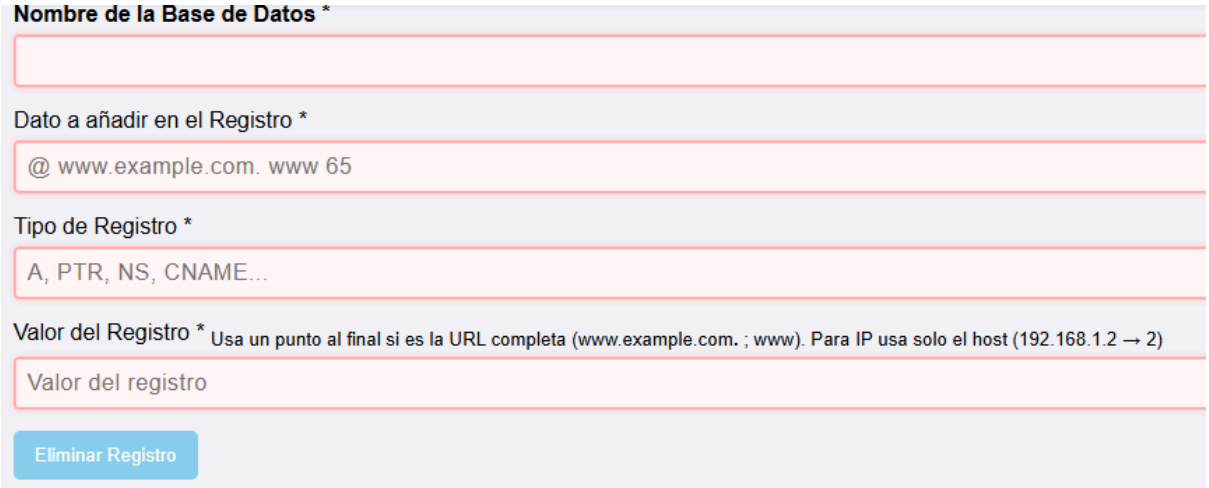
Se puede establecer una configuración, aunque por defecto ya viene una establecida.



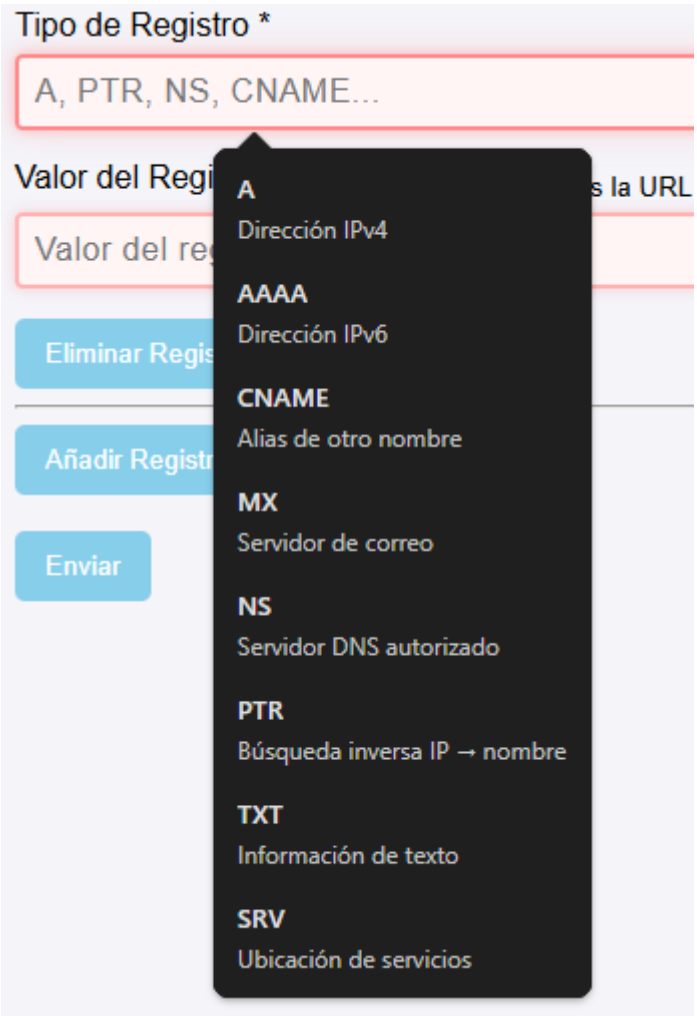
Un nameserver autoritativo es el servidor que contiene la información oficial de una zona DNS. Si alguien pregunta por www.empresa.local, el servidor autoritativo es empresa.local, este se identifica mediante un registro NS, pero se puede establecer aquí.



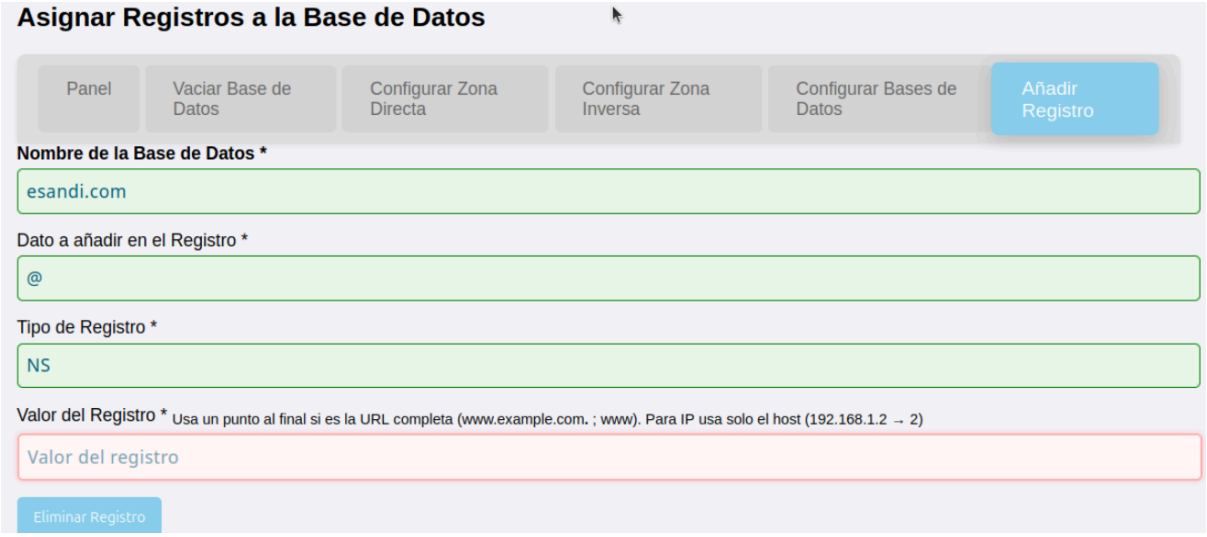
Después se debe introducir, de forma opcional, la dirección IP de la zona. Y finalmente se crea.



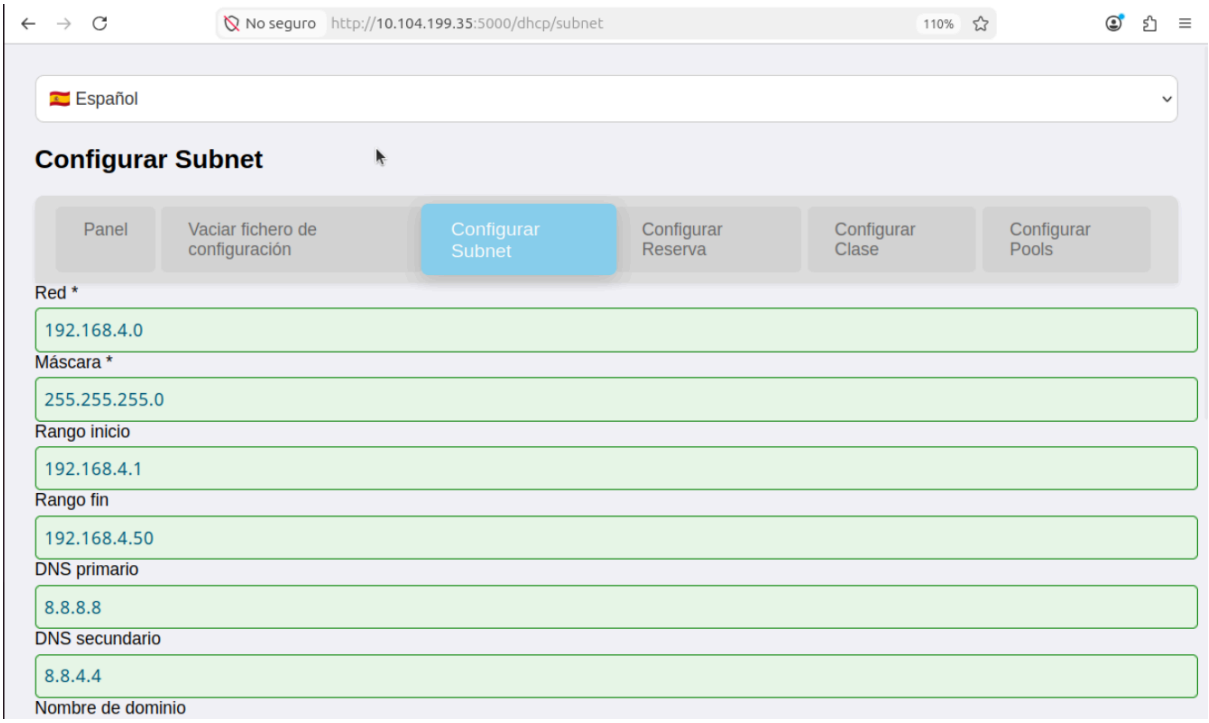
A la hora de agregar registros, se debe establecer el nombre de la base de datos, el dato a añadir siendo este una dirección web, una dirección IP, etc, después el tipo de registro, y finalmente el valor. Se explica que se debe agregar un punto al final si es una URL completa, mientras que para una dirección IP sólo se establece el host.



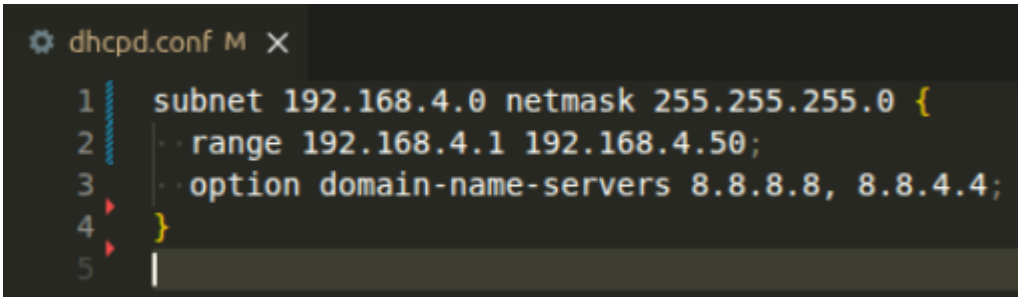
También indica los tipos de registros que se pueden agregar con una breve explicación.



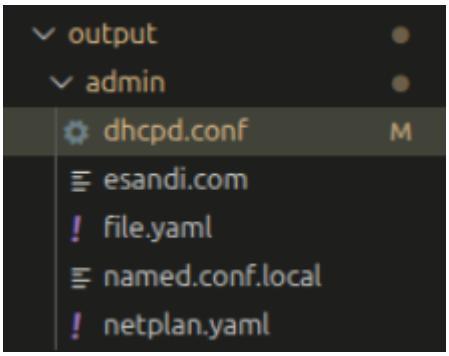
Ejemplo de la configuración de DNS



Otro ejemplo de configuración, pero en este caso DHCP

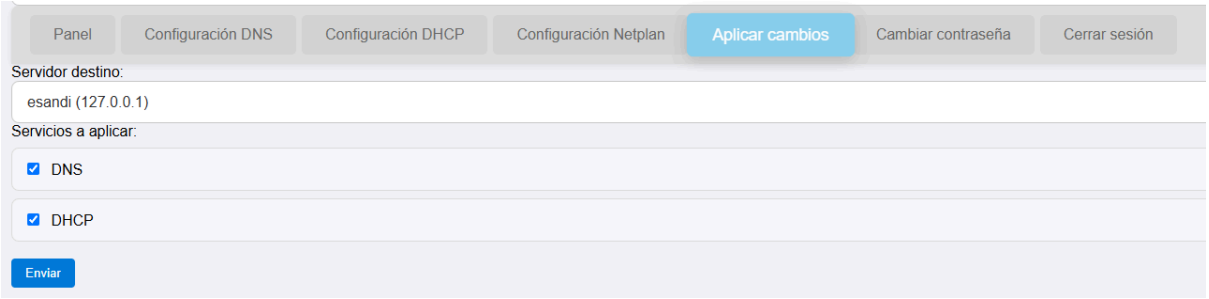


Así quedaría el fichero de configuración final



Se encuentra en esta ruta.

Ahora si lo quiero enviar al servidor:

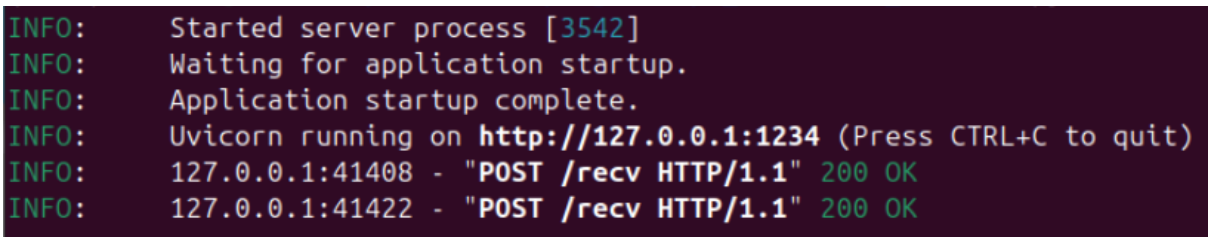


En la parte de aplicar cambios se selecciona el servicio que se va a reiniciar.

- En el caso de que se envíe el fichero de DNS, se enviarán los contenidos relacionados con DNS
- En el caso de que sea DHCP, se enviará el fichero de configuración y se realizará una verificación de errores en el caso de que no funcione correctamente.

LIMBONET verifica que la configuración es válida usando las herramientas de comprobación proporcionadas por los servicios

- named-checkconf
- named-checkzone
- dhcpd -t



El servidor agente del nodo recibe la configuración, valida mediante API Key y aplica los cambios en el sistema local



El sistema registra la recepción del comando en el servicio destino

ID	Usuario	Servidor (ID)	Comando	Fecha	Fichero	Dirección IP
1	admin	3	MOVER FICHERO "esandi.com" A "DNS"	2026-03-19 16:10:23	esandi.com	127.0.0.1
2	admin	3	MOVER FICHERO "esandi.com" A "DNS"	2026-03-19 16:10:23	None	127.0.0.1
3	admin	3	MOVER FICHERO "dhcpd.conf" A "DHCP"	2026-03-31 10:25:54	dhcpd.conf	127.0.0.1
4	admin	3	MOVER FICHERO "dhcpd.conf" A "DHCP"	2026-03-31 10:25:54	None	127.0.0.1

También se guarda registro en los logs de esa acción.
De igual manera, se puede verificar el estado del servicio DNS y DHCP para verificar si hay errores.

Backend

El WebGUI está implementado en Flask, y puede desplegarse en sistemas Linux o Windows Server

```
csrf = CSRFProtect()
csrf.init_app(app)
```

Usa CSRFProtect para la protección contra ataques CSRF

```
hashed_password = hashpw(contraseña.encode('utf-8'), gensalt())
cursor.execute("INSERT INTO usuarios (usuario, contraseña, rol)
conn.commit()
conn.close()
flash("User created successfully", "success")
return redirect(url_for("panel.panel_home"))
```

Las contraseñas se guardan tras haberse pasado por un algoritmo de hash (Bcrypt) con sal.

```
conexion_db = sqlite3.connect("database.db")
cursor_db = conexion_db.cursor()
logs = [list(log) for log in logs]

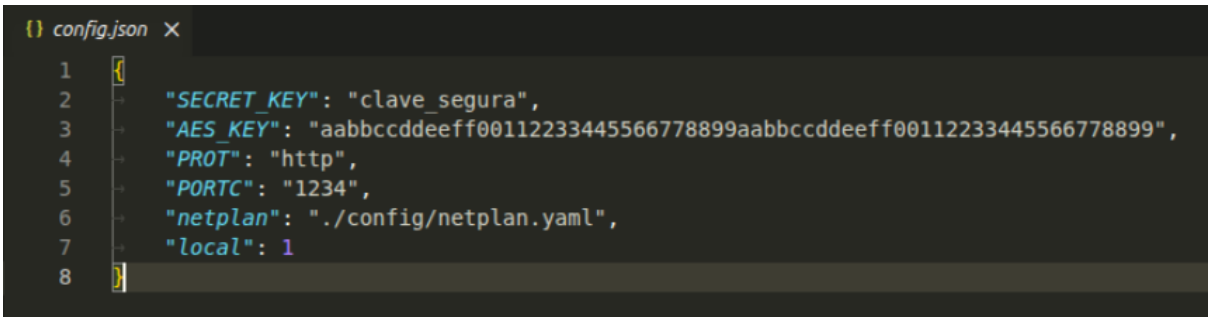
for i, log in enumerate(logs):
    cursor_db.execute("SELECT ip FROM dispositivos WHERE Iddispo = ?", (log[2],))
    ip = cursor_db.fetchone()

    log.append(ip[0] if ip else "IP no encontrada")
conexion_db.close()
```

Las consultas SQL están parametrizadas para evitar inyección SQL

```
cursor.execute("""
    SELECT d.ip
    FROM dispositivos d
    JOIN permisos p ON p.id_dispositivo = d.iddispo
    JOIN usuarios u ON u.Idusuario = p.id_usuario
    WHERE u.usuario = ? AND p.permiso = 2 AND d.iddispo = ?
""", (session["usuario"], servidor_id))
row = cursor.fetchone()
conn.close()
if not row:
    flash("Unauthorized attempt to access a server.", "danger")
    return redirect(url_for("netplan_bp.configurarnetplan"))
ip_servidor = row[0]
```

En varias partes del código se verifican los permisos para evitar accesos no autorizados a los servidores a base de manipulación de datos transmitidos entre frontend y backend



El fichero de configuración de config.json tiene los siguientes campos

- SECRET_KEY: clave segura del servidor
- AES_KEY: clave de cifrado AES (actualmente no utilizada)
- PROT: protocolo, si es HTTP o HTTPS (recomendable)
- PORTC: puerto de acceso
- Netplan: Ruta del fichero de configuración de red del S.O.(uso limitado a entornos de laboratorio o virtualización)
- Local: si se establece a 1, se usará el modo local, si está en 0, no se usará.

El modo local está pensado para entornos de prueba o laboratorio, permitiendo que el Web GUI aplique configuraciones directamente sobre el sistema local sin necesidad de comunicación con nodos externos.

{} localconfig.json

```
{
    "TEMP_DIR": "./output/simulated/tmp",
    "DHCP_DIR": "./output/simulated/dhcp",
    "DNS_DIR": "./output/simulated/dns",
    "NETPLAN_DIR": "./output/simulated/netplan"
}
```

La configuración de localconfig.json sirve para especificar las carpetas de la respectiva configuración.

app.py M

```
from flask import Flask, render_template, abort
from blueprints import blueprints, blueprints_local
from flask_wtf.csrf import CSRFProtect
import json

with open ("./config/config.json", "r") as f:…

app = Flask(__name__)
app.config["SECRET_KEY"] = data["SECRET_KEY"]
app.config["AES_KEY"] = bytes.fromhex(data["AES_KEY"])
app.config["PROT"] = data["PROT"]
app.config["PORTC"] = data["PORTC"]

if data["local"] == 0:…
else:…

csrf = CSRFProtect()
csrf.init_app(app)

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/index")
def index():
    return render_template("home.html")
```

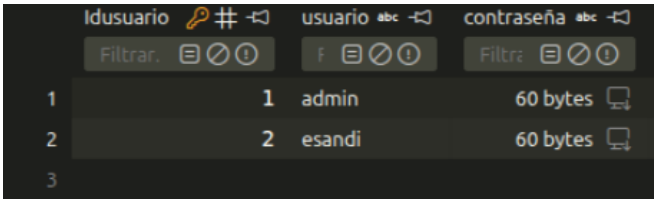
El archivo app.py carga CSRF y también los blueprints necesarios para que funcione correctamente.

La base de datos se compone de lo siguiente:
En el caso de la base de datos database.db: tiene tres tablas

Filas: 1 — Filtrar 1 filas...

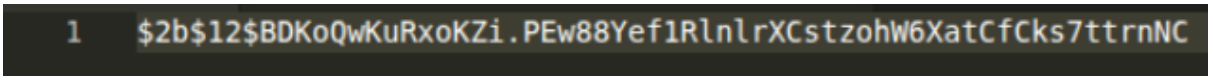
	Iddispo	hostname	ip	api_key
1	3	esandi	127.0.0.1	pon_tu_api_key_aqui
2				

Tabla dispositivos: almacena nodos registrados en el sistema



	Idusuario	usuario	contraseña
1	1	admin	60 bytes
2	2	esandi	60 bytes
3			

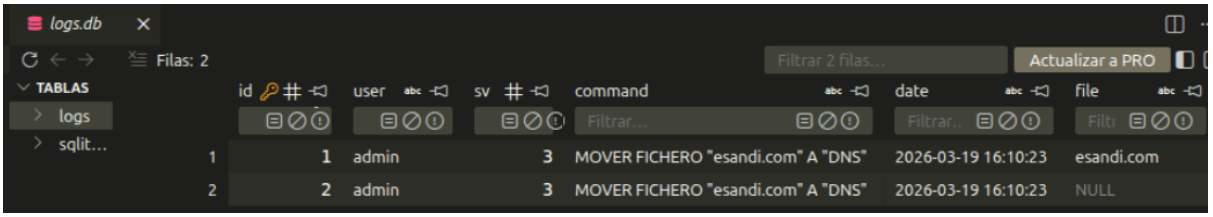
Y la tabla de usuarios, que registra al usuario y su contraseña



1	$2b$12$BDKoQwKuRxoKZi.PEw88Yef1RlnlrXCstzohW6XatCfCks7ttrnNC

Para mayor seguridad, las contraseñas están hasheadas.

En el caso de la base de datos de logs.db:



logs.db — Filas: 2 — Filtrar 2 filas... — Actualizar a PRO

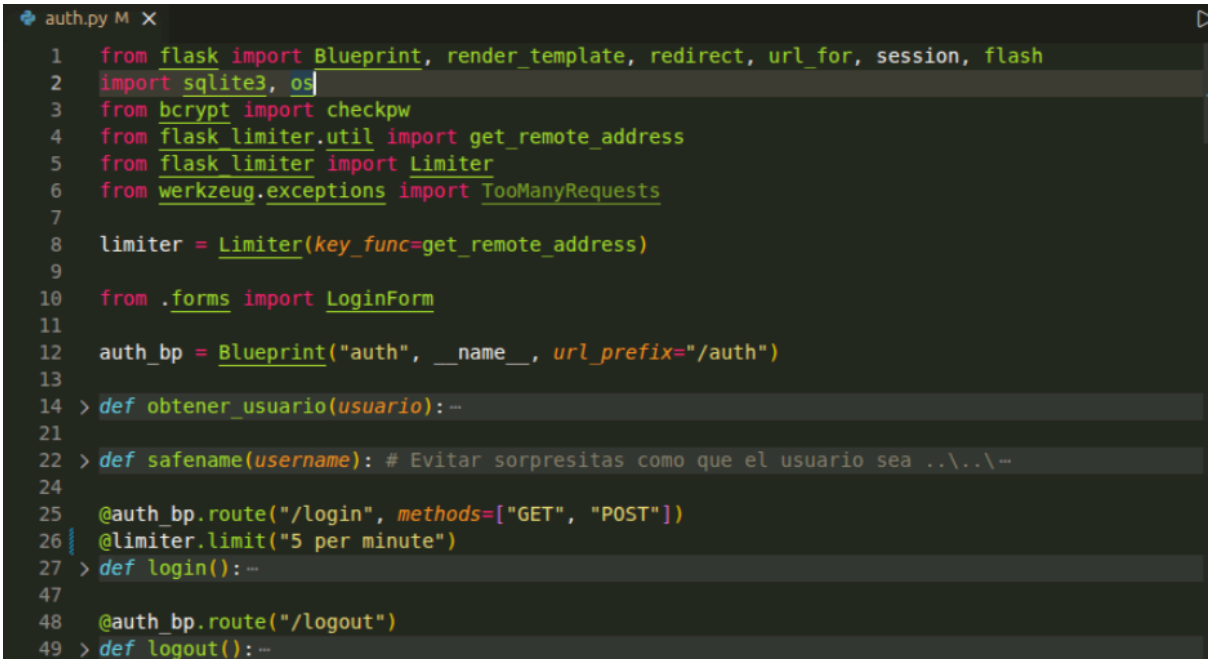
TABLAS: logs, sqlit...

	id	user	sv	command	date	file
1	1	admin	3	MOVER FICHERO "esandi.com" A "DNS"	2026-03-19 16:10:23	esandi.com
2	2	admin	3	MOVER FICHERO "esandi.com" A "DNS"	2026-03-19 16:10:23	NULL

Solo se compone de una tabla, que almacena los datos de las interacciones de los usuarios con los servidores

Auth.py

Este Blueprint, como su nombre indica, se encarga de autenticar al usuario



auth.py M

```python
from flask import Blueprint, render_template, redirect, url_for, session, flash
import sqlite3, os
from bcrypt import checkpw
from flask_limiter.util import get_remote_address
from flask_limiter import Limiter
from werkzeug.exceptions import TooManyRequests

limiter = Limiter(key_func=get_remote_address)

from .forms import LoginForm

auth_bp = Blueprint("auth", __name__, url_prefix="/auth")

def obtener_usuario(usuario):…

def safename(username): # Evitar sorpresitas como que el usuario sea ..\..\…

@auth_bp.route("/login", methods=["GET", "POST"])
@limiter.limit("5 per minute")
def login():…

@auth_bp.route("/logout")
def logout():…
```

Solo tiene dos rutas, login y logout. Para registrarse, lo debe hacer un administrador.

```python
@auth_bp.route("/login", methods=["GET", "POST"])
@limiter.limit("5 per minute")
def login():
    form = LoginForm()

    if form.validate_on_submit():
        user = form.usuario.data
        pwd = form.contraseña.data
        datos = obtener_usuario(user)

        if datos and pwd:
            _id, usuario, contraseña_hash, rol = datos
            if checkpw(pwd.encode(), contraseña_hash):
                session.clear()
                session["usuario"] = usuario
                session["rol"] = rol
                flash("Login successful", "success")
                os.makedirs("./output/" + safename(session["usuario"]), exist_ok=True)

                return redirect(url_for("panel.panel_home"))
        flash("Incorrect username or password", "danger")
    return render_template("login.html", form=form)
```

Ruta de inicio de sesión.

Los siguientes apartados contienen endpoints que no aplican cambios directamente en el sistema, genera solicitudes que serán procesadas por el servicio agente del nodo correspondiente

Dhcp_bp.py

dhcp_bp.py M

```python
from flask import Blueprint, render_template, request, redirect, url_for, flash, session, current_app
from functools import wraps
from modules.dhcp import formatDHCP, subnet as crear_subnet, reserva as crear_reserva, claseMAC, claseID, pool

dhcp_bp = Blueprint("dhcp", __name__, url_prefix="/dhcp")

def login_required_dhcp(f):…

@dhcp_bp.route("/")
@login_required_dhcp
def dhcp_home():…

@dhcp_bp.route("/vaciar")
@login_required_dhcp
def vaciar_config():…

@dhcp_bp.route("/subnet", methods=["GET", "POST"])
@login_required_dhcp
def subnet():…

@dhcp_bp.route("/reserva", methods=["GET", "POST"])
@login_required_dhcp
def reserva_dhcp():…

@dhcp_bp.route("/clase", methods=["GET", "POST"])
@login_required_dhcp
def clase_dhcp():…

@dhcp_bp.route("/pool", methods=["GET", "POST"])
@login_required_dhcp
def pool_dhcp():…
```

Este fichero se encarga de la configuración de los servicios DHCP, haciendo un POST en el caso de que se quiera mandar información

```python
def pool_dhcp():
    if request.method == "POST":
        red = request.form.get("red")
        mascara = request.form.get("mascara")
        rango1 = request.form.get("rango1")
        rango2 = request.form.get("rango2")
        dns1 = request.form.get("dns1")
        dns2 = request.form.get("dns2")
        dnsname = request.form.get("dnsname")
        subnetmask = request.form.get("subnetmask")
        dg = request.form.get("dg")
        broadcast = request.form.get("broadcast")
        lease = request.form.get("lease")
        maxlease = request.form.get("maxlease")

        with open("./output/"+ session["usuario"] + "/dhcpd.conf", "a") as f: # SIN el } final
            f.write(crear_subnet(red, mascara, rango1, rango2, dns1, dns2, dnsname, subnetmask, dg, broadcast, lea

        # --- Clases dentro del pool ---
        clase_nombres = request.form.getlist("clase_nombre[]")
        clase_acciones = request.form.getlist("clase_accion[]")
        clase_rango1 = request.form.getlist("clase_rango1[]")
        clase_rango2 = request.form.getlist("clase_rango2[]")
        clase_dns1 = request.form.getlist("clase_dns1[]")
        clase_dns2 = request.form.getlist("clase_dns2[]")
        clase_dnsname = request.form.getlist("clase_dnsname[]")
        clase_subnetmask = request.form.getlist("clase_subnetmask[]")
        clase_dg = request.form.getlist("clase_dg[]")
        clase_broadcast = request.form.getlist("clase_broadcast[]")
        clase_lease = request.form.getlist("clase_lease[]")
        clase_maxlease = request.form.getlist("clase_maxlease[]")

        with open("./output/"+ session["usuario"] + "/dhcpd.conf", "a") as f:
            for i in range(len(clase_nombres)):
                f.write(pool(
```

Se puede acceder vía POST o GET, se usa endpoints GET para consulta de los datos y POST para la actualización de la configuración

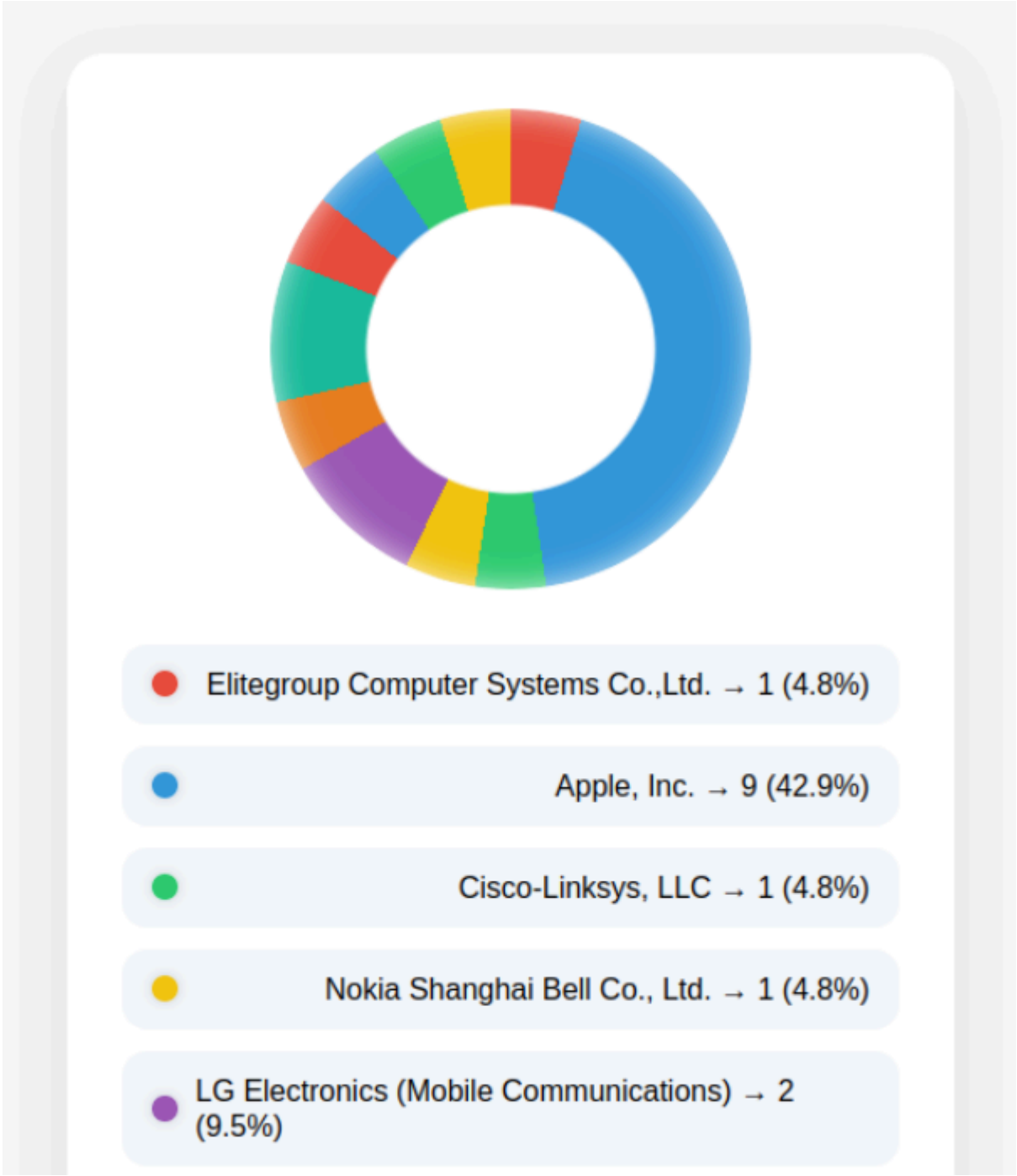
Resultados

ID	IP	MAC	Hostname	STARTS	ENDS	CLTT	BINDING STATE	DATA
3	192.168.4.200	bc:24:11:87:cd:85	esandi	2026/05/12 08:41:28	2026/05/12 08:51:28	2026/05/12 08:41:28	active	Ver datos
2	192.168.4.200	bc:24:11:87:cd:85	None	2026/05/12 08:37:58	2026/05/12 08:38:33	2026/05/12 08:37:58	free	Ver datos
1	192.168.4.200	bc:24:11:87:cd:85	esandi	2026/05/12 08:37:58	2026/05/12 08:47:58	2026/05/12 08:37:58	active	Ver datos

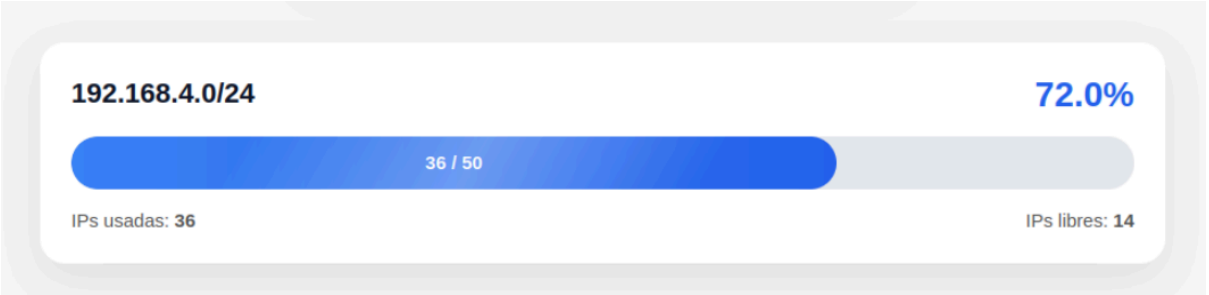
También tiene una opción para ver los registros de las concesiones.

```
lease 192.168.4.200 {
  starts 2 2026/05/12 08:41:28;
  ends 2 2026/05/12 08:51:28;
  cltt 2 2026/05/12 08:41:28;
  binding state active;
  next binding state free;
  rewind binding state free;
  hardware ethernet bc:24:11:87:cd:85;
  client-hostname "esandi";
}
```

Y permite ver de forma más detallada el registro.
Esto se consigue mediante una consulta SQL a la base de datos que contiene los registros..

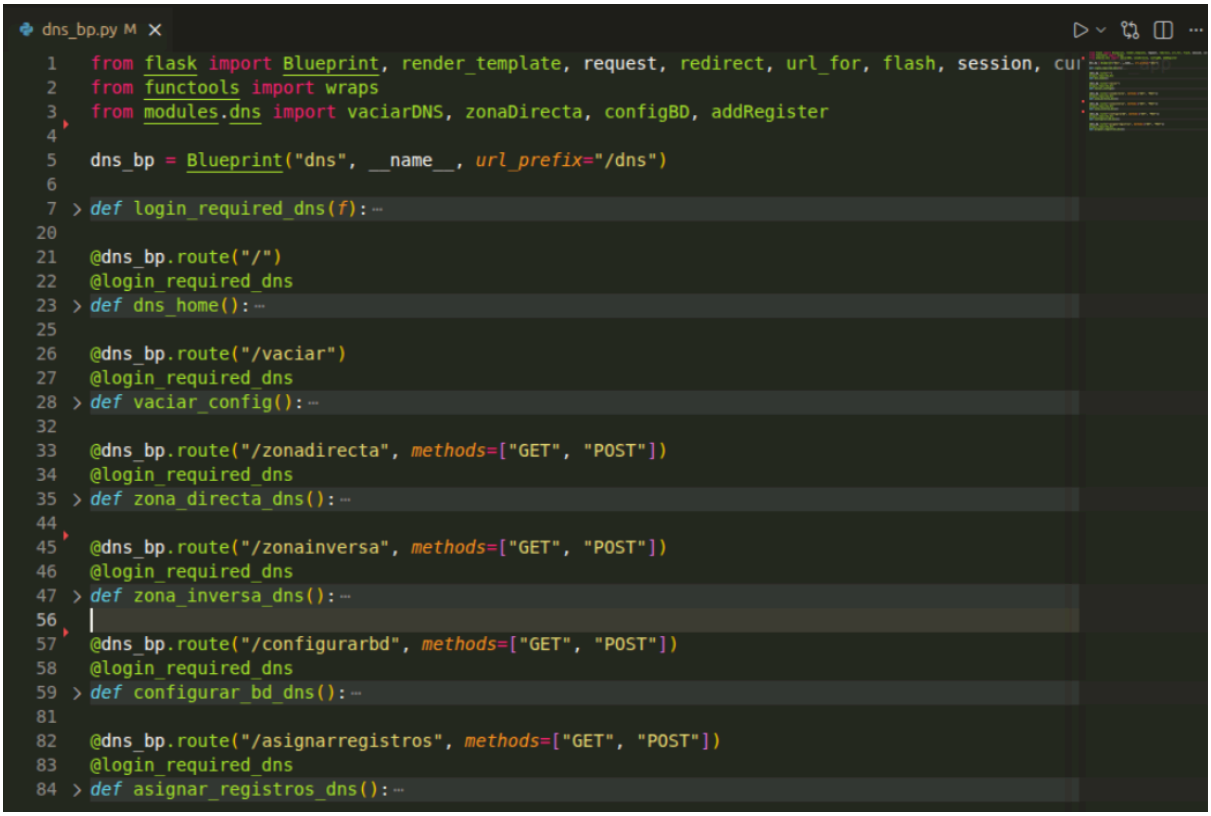


También ofrece un gráfico que muestra información de los vendedores de las direcciones MAC a la cual se han asignado direcciones IP.



Y por cada subnet, cuántas direcciones IP libres y vacías quedan

Dns_bp.py



Funciona de la misma manera que el blueprint anterior, pero esta vez funciona con los registros DNS

Local.py

Solo disponible si se activa el modo local en el fichero, usado para aplicar los cambios en el servidor



```python
from flask import Blueprint, render_template, abort, request, flash, redirect, url_for, curre
import json, re, subprocess, os, sqlite3
from functools import wraps

with open ("./config/localconfig.json", "r") as f:
    datos = json.load(f)

ALLOWED_COMMANDS = ["ACTUALIZAR_DNS", "LIMPIAR_CACHE"]
TEMP_DIR = datos["TEMP_DIR"]

ubicaciones = {
    "TMP": datos["TEMP_DIR"],
    "DHCP": datos["DHCP_DIR"],
    "DNS": datos["DNS_DIR"],
    "NETPLAN": datos["NETPLAN_DIR"]
}

def procesar(texto):…

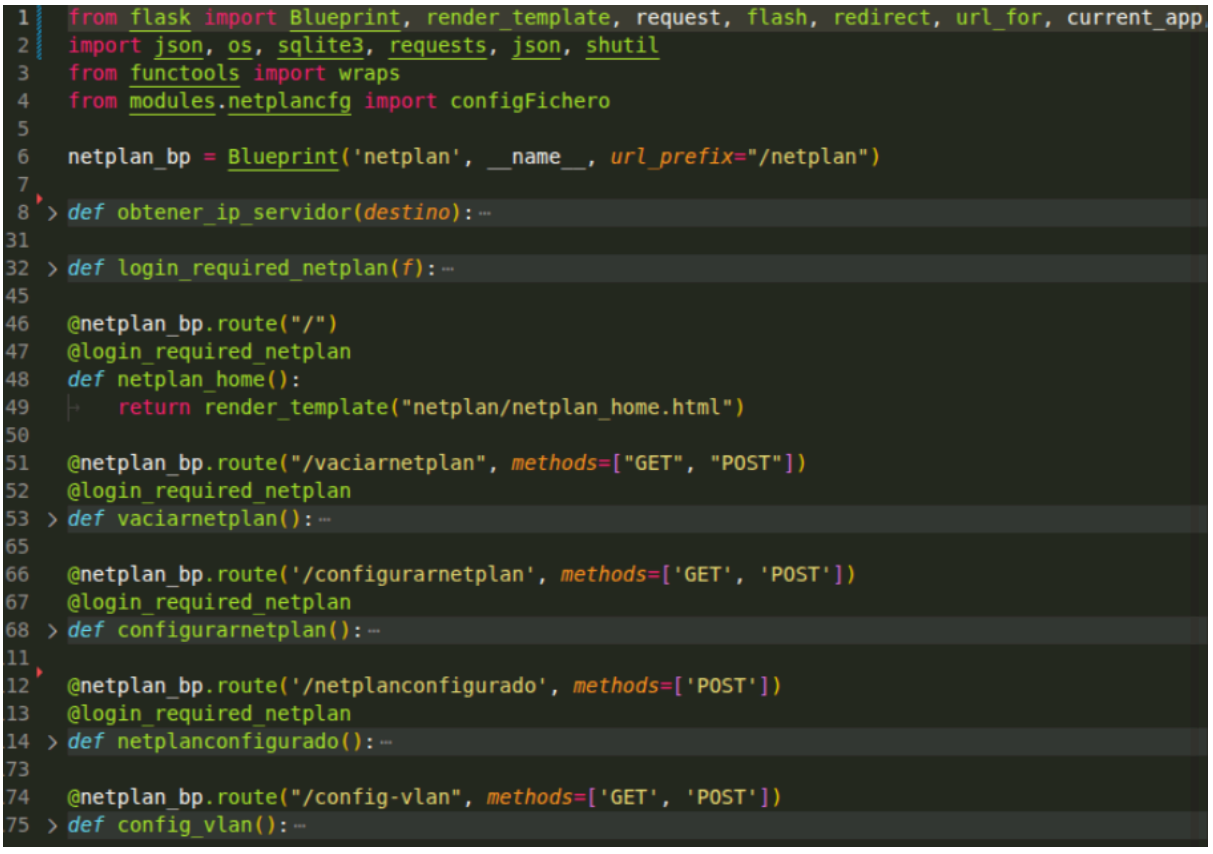
local_bp = Blueprint("local", __name__, url_prefix="/")

def login_required(f):…

@local_bp.route("/localmode")
@login_required
def localmode():…

@local_bp.route("/confirmlocal", methods=["POST"])
@login_required
def confirmlocal():…
```

Netplan_bp.py



```python
from flask import Blueprint, render_template, request, flash, redirect, url_for, current_app
import json, os, sqlite3, requests, json, shutil
from functools import wraps
from modules.netplancfg import configFichero

netplan_bp = Blueprint('netplan', __name__, url_prefix="/netplan")

def obtener_ip_servidor(destino):…

def login_required_netplan(f):…

@netplan_bp.route("/")
@login_required_netplan
def netplan_home():
    return render_template("netplan/netplan_home.html")

@netplan_bp.route("/vaciarnetplan", methods=["GET", "POST"])
@login_required_netplan
def vaciarnetplan():…

@netplan_bp.route('/configurarnetplan', methods=['GET', 'POST'])
@login_required_netplan
def configurarnetplan():…

@netplan_bp.route('/netplanconfigurado', methods=['POST'])
@login_required_netplan
def netplanconfigurado():…

@netplan_bp.route("/config-vlan", methods=['GET', 'POST'])
def config_vlan():…
```

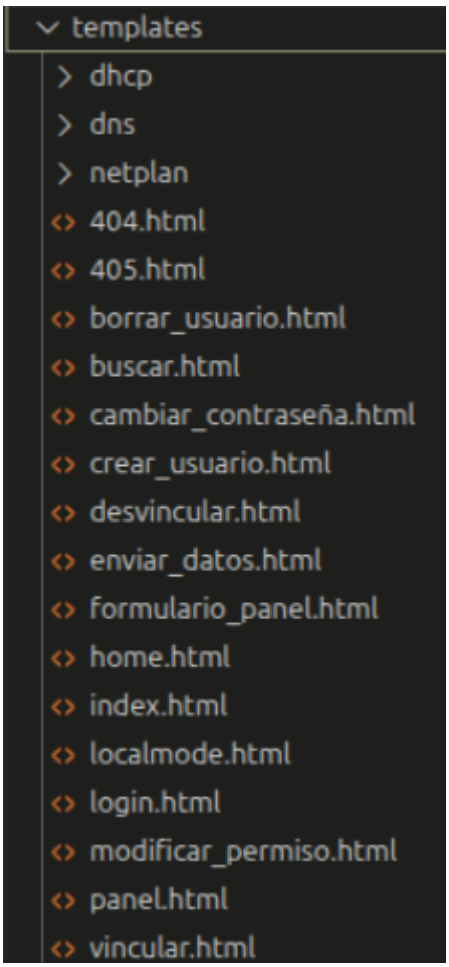
Funciona igual que el blueprint de DHCP y DNS, pero con la parte de Netplan. Se comenta que se debe evitar modificar interfaces activas en producción ya que puede provocar pérdida de conectividad.

panel.py

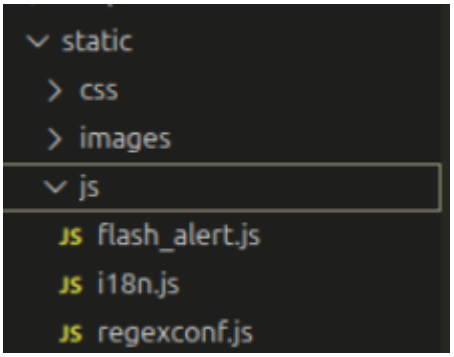


```
41
42  @panel_bp.route("/", methods=["GET", "POST"])
43  @login_required(rol_minimo=0)
44  def panel_home():
88
89  @panel_bp.route("/change_password", methods=["GET", "POST"])
90  @login_required(rol_minimo=0)
91  def change_password():
124
125 class FormularioRegistroUsuario(FlaskForm):
131
132 @panel_bp.route("/crear_usuario", methods=["GET", "POST"])
133 @login_required(rol_minimo=4)  # Verifica que el rol del usuario sea 4 o mayor
134 def crear_usuario():
157
158 @panel_bp.route("/borrar_usuario", methods=["GET", "POST"])
159 @login_required(rol_minimo=4)
160 def borrar_usuario():
190
191 @panel_bp.route("/vincular", methods=["GET", "POST"])
192 @login_required(rol_minimo=3)
193 def vincular():
234
235 def obtener_api(id_dispo):
244
245 @panel_bp.route("/desvincular", methods=["GET", "POST"])
246 @login_required(rol_minimo=4)
247 def desvincular():
289
```

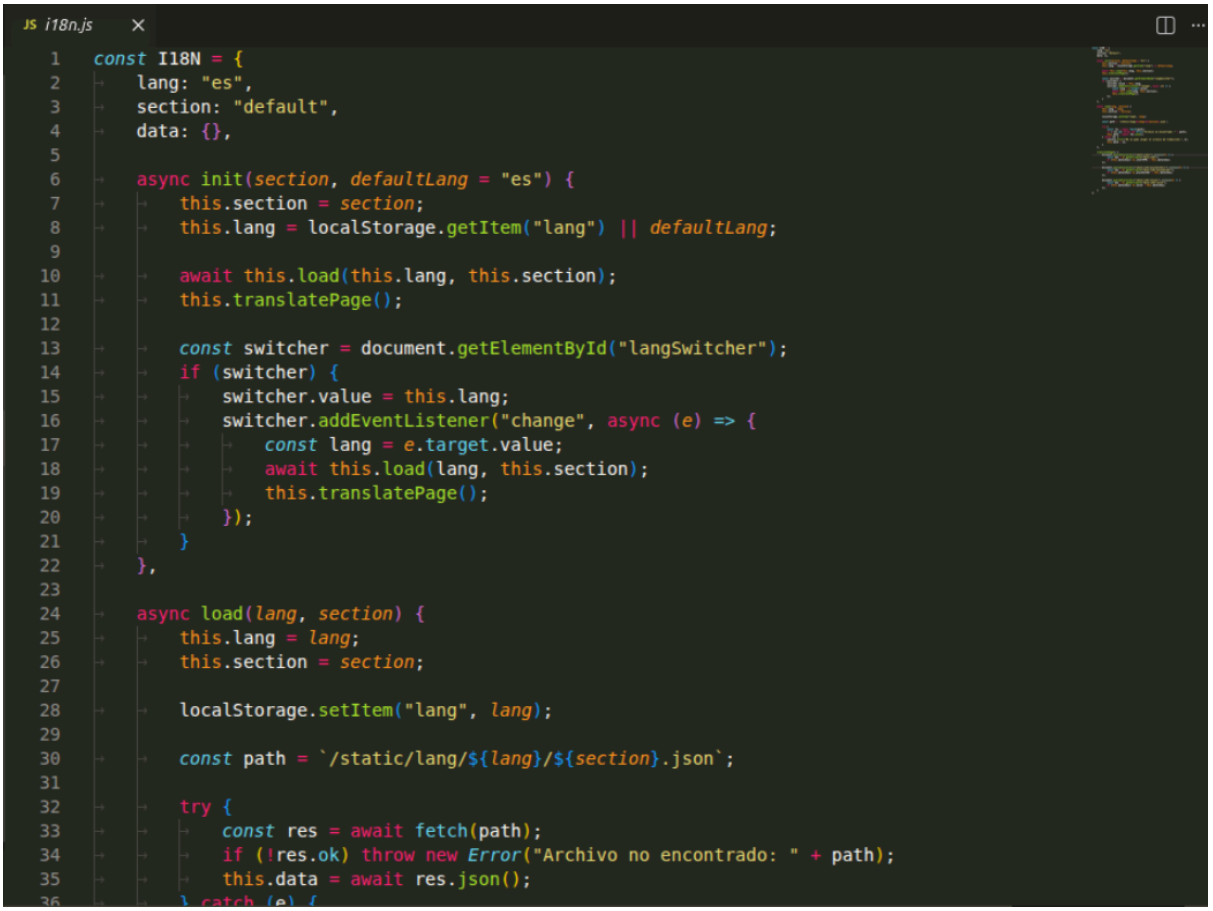
En este caso, se configuran funciones de dentro del panel, tanto como las de cambiar contraseña, como las de enviar datos a otros servidores



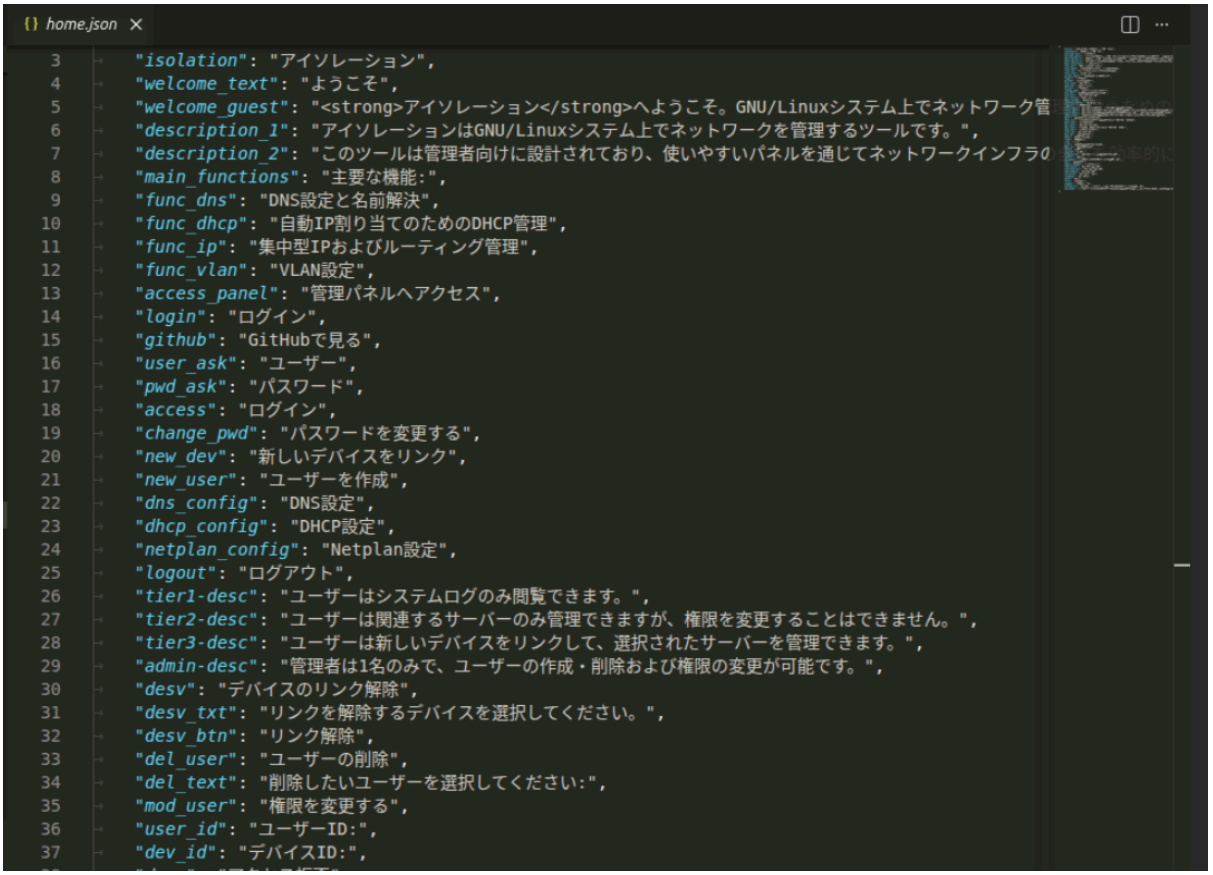
Los ficheros HTML se guardan en la carpeta templates



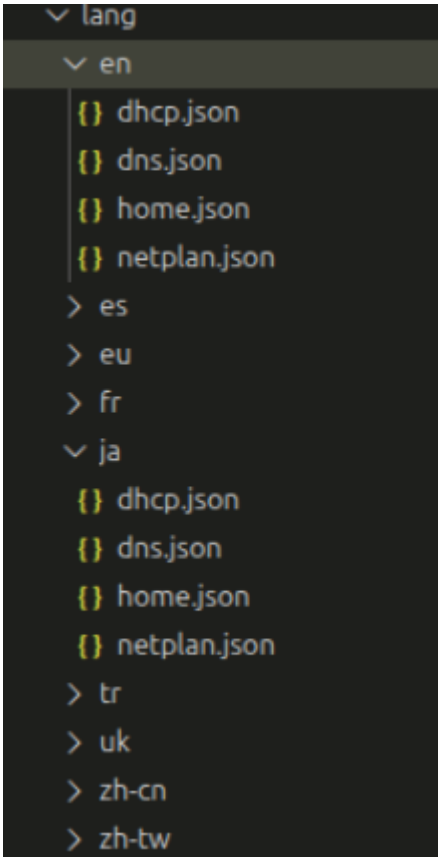
Se usan en total tres ficheros JavaScript externos



Este fichero JavaScript se encarga de traducir LIMBONET a diferentes idiomas



Este es el JSON de la traducción al japonés



Los ficheros se guardan en la carpeta lang perteneciente a static

Diseño de Limbonet service

```
INFO:     Started server process [3618]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:1234 (Press CTRL+C to quit)
```

Primero se ejecuta con FastAPI. Se recomienda el uso de HTTPS para cifrar la comunicación entre el WebGUI y el servicio agente

```
{
    "api_key":"pon_tu_api_key_aqui",
    "protocol": "http",
    "PORT": "1234",
    "TEMP_DIR": "./tmp",
    "DHCP_DIR": "./dhcp",
    "DHCP_LEASES": "./dhcp/dhcpd.leases",
    "DNS_DIR": "./dns",
    "NETPLAN_DIR": "./netplan",
    "dhcp_monitor": "yes",
    "email": "example@email.com",
    "password": "password",
    "emaild": "example@email.com",
    "smtp_server": "smtp.gmail.com",
    "smtp_port": "587",
    "dhcp_threshold": "3",
    "subnet_threshold": "80"

}
```

El archivo config.json define las reglas, con los siguientes campos:

- API_KEY: una clave de acceso a la API
- PORT: Puerto de acceso
- El resto son las direcciones que suelen estar en /etc/, se dejan en una carpeta local para realizar las pruebas

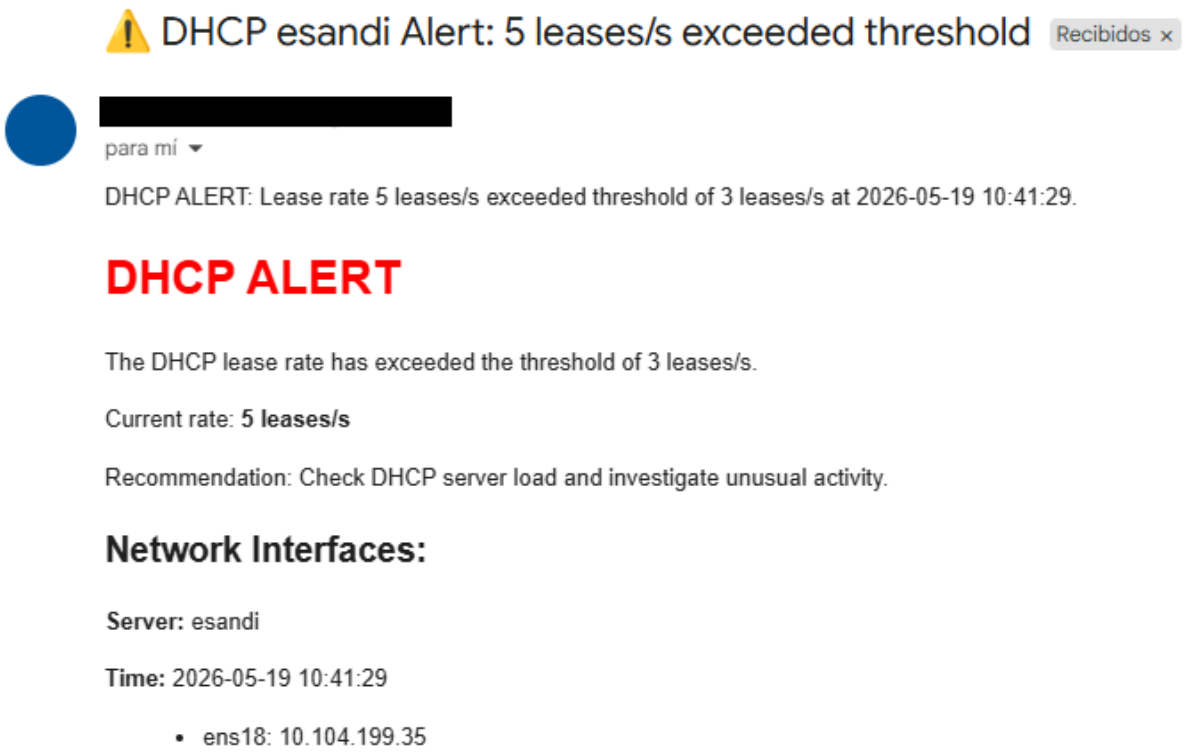
También están los siguientes campos:

- Email: Para introducir una dirección de correo electrónico desde la cual se enviarán notificaciones
- Password: La contraseña de la cuenta
- Email: La dirección de correo electrónico de destino
- Smtp_server: Dirección del servidor SMTP
- Smtp_port: Puerto del servidor SMTP

Y respecto a DHCP

- Dhcp_threshold: Un umbral para notificar por correo electrónico cuando se supera una cantidad de leases por segundo

- Subnet_threshold: Un umbral para notificar por correo electrónico cuando un porcentaje de direcciones IP usadas se supera
- Dhcp_monitor: Activa o desactiva la monitorización de DHCP



⚠️ DHCP esandi Alert: 5 leases/s exceeded threshold Recibidos ×

[redacted]

para mí

DHCP ALERT: Lease rate 5 leases/s exceeded threshold of 3 leases/s at 2026-05-19 10:41:29.

DHCP ALERT

The DHCP lease rate has exceeded the threshold of 3 leases/s.

Current rate: **5 leases/s**

Recommendation: Check DHCP server load and investigate unusual activity.

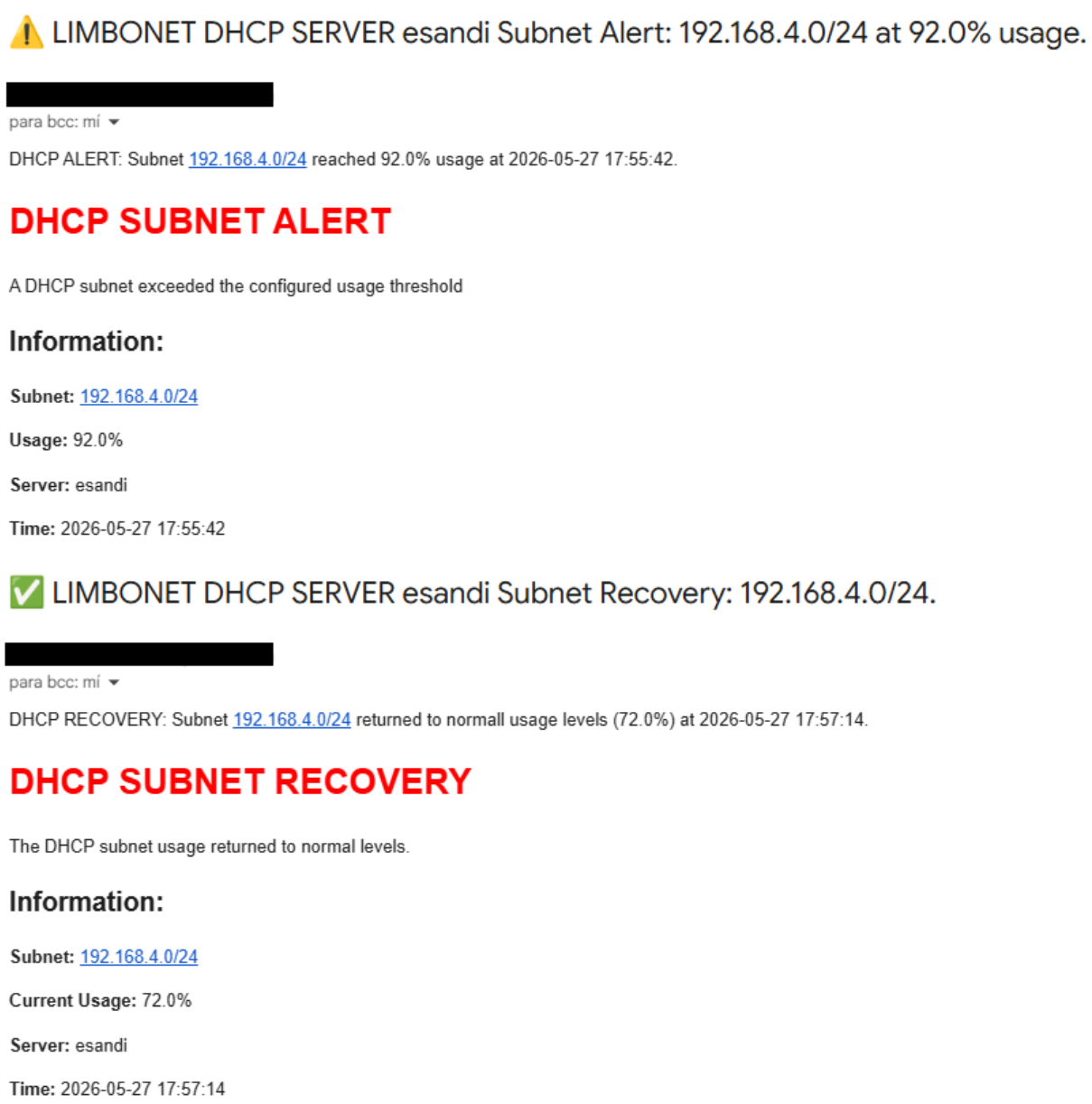
Network Interfaces:

Server: esandi

Time: 2026-05-19 10:41:29

- ens18: 10.104.199.35

Ejemplo de correo recibido cuando se supera una cantidad de leases por segundo,



Ejemplo de correos cuando se alcanza el umbral

Una vez configurado, el servidor se inicia. Lo ideal es que esté escuchando en una red local no accesible por los demás.

```python
from fastapi import APIRouter, HTTPException, Header
from pydantic import BaseModel
import os, re, subprocess, base64, json
from typing import Optional
import datetime

with open ("./config.json", "r") as f:
    datos = json.load(f)

ALLOWED_COMMANDS = ["ACTUALIZAR_DNS", "LIMPIAR_CACHE"]
TEMP_DIR = datos["TEMP_DIR"]

ubicaciones = {…

router = APIRouter()

def verify_api_key(x_api_key: str = Header(...)):
    if x_api_key != datos["api_key"]:
        raise HTTPException(status_code=401, detail="Unauthorized")

class CommandPayload(BaseModel):
    data: str
    filename: Optional[str] = None
    #timestamp: float

def procesar(texto):…

@router.post("/recv")
def recv(payload: CommandPayload, x_api_key: str = Header(...)):…
```

El fichero commands.py se basa en una función para "procesar" los comandos.
Los comandos están limitados a operaciones controladas definidas previamente en el sistema, para evitar la ejecución arbitraria de comandos no autorizados.
La URL /recv se encarga de recibir los comandos y los ficheros recibidos, y procesarlos.
Únicamente procesa solicitudes autenticadas mediante la API Key válida

```python
from fastapi import APIRouter, HTTPException, Header, Depends
import socket, json, sqlite3
from pydantic import BaseModel
import os, base64
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

router = APIRouter()
with open ("./config.json", "r") as f:
    datos = json.load(f)

def verify_api_key(x_api_key: str = Header(...)):
    if x_api_key != datos["api_key"]:
        raise HTTPException(status_code=401, detail="Unauthorized")

@router.get("/status")
def status():
    return {"status": "alive"}

@router.post("/hostname")
def get_hostname(api_key: str = Depends(verify_api_key)):...

class RemoveDeviceReq(BaseModel):...

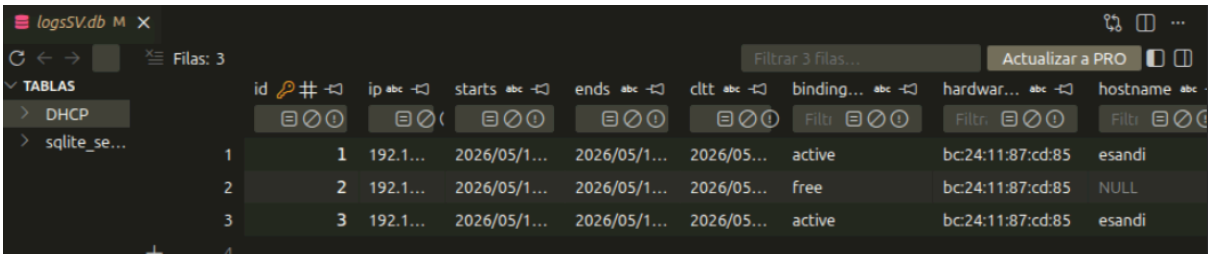
@router.post("/remove-disp")
def remove_disp(req: RemoveDeviceReq, api_key: str = Depends(verify_api_key)):...

@router.get("/interfaces")
def interfaces(req: RemoveDeviceReq, api_key: str = Depends(verify_api_key)):...
```

En status.py hay URL que devuelven información del servidor, en este caso si está online, el hostname, para eliminar el dispositivo, y las interfaces.



La base de datos, que es databaseSRV.db, contiene una tabla de control de acceso de nodos autorizados



Otra base de datos es logsSV.db que contiene los registros de concesiones de DHCP. Limbonet service se encarga de transformar los registros a una base de datos SQL.



Esto es posible debido a que se almacena el inodo del fichero de leases, y en el caso de que haya más, no recorre el documento entero, así evitando los duplicados en la base de datos. Esto para permitir una búsqueda más eficiente.